



本教學投影片係屬教科書著作之延伸，亦受著作權法之保護。

請依出版社授權使用規範利用本教學資源，非經授權許可不得以影印、拷貝、列印等方法進行重製，亦不得改作、公開展示著作內容，亦請勿對第三人公開傳輸、散布。



Chapter 4

Combinational Logic

Prof. Chih-Cheng Tseng (曾志成)

cctseng@mail.ntou.edu.tw

Chapter Objectives (1/2)

- Given its logic diagram, know how to analyze a combinational logic circuit.
- Understand the functionality of a half adder and a full-adder.
- Understand the concepts of overflow and underflow.
- Understand the implementation of a binary adder.
- Understand the implementation of a binary coded decimal (BCD) adder.
- Understand the implementation of a binary multiplier.
- Understand fundamental combinational logic circuits: decoder, encoder, priority encoder, multiplexer, and three-state gate.
- Know how to implement a Boolean function with a multiplexer.
- Understand the distinction between gate-level, dataflow, and a behavioral modeling with hardware description languages (HDLs).

Chapter Objectives (2/2)

- Be able to write a gate-level Verilog or VHDL model of a fundamental logic circuit.
- Be able to write a hierarchical HDL model of a combinational logic circuit.
- Be able to write a dataflow model of a fundamental combinational logic circuit.
- Be able to write a Verilog continuous assignment statement or a VHDL signal assignment statement.
- Know how to declare a Verilog procedural block, or a VHDL process.
- Be able to write a simple testbench.

4.1 Introduction

- Logic circuits for digital systems may be combinational or sequential.
 - Combinational circuit
 - consists of logic gates
 - outputs at any time are determined from only the present combination of inputs.
 - Sequential circuit
 - contains memory elements and logic gates
 - the outputs are a function of the current inputs and the state of the memory elements
 - the outputs also depend on past inputs
 - the outputs are changed only at a specific time
 - positive/negative **edge**-trigger
 - positive/negative **level**-trigger

4.2 Combinational Circuits

- A combinational circuits
 - 2^n possible combinations of input values

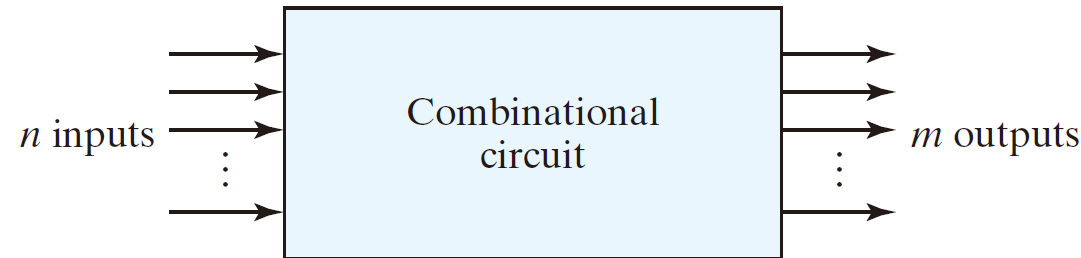


FIGURE 4.1
Block diagram of combinational circuit

- Specific functions
 - Adders, subtractors, comparators, decoders, encoders, and multiplexers
 - Medium-scale integrated (MSI) circuits or standard cells in complex very large-scale integrated (VLSI) circuits such as application-specific integrated circuits (ASICs).

4.3 Analysis of Combinational Circuit (1/5)

- Analysis of a combinational circuit determines its functionality, that is the logic function that the circuit implements.
- Steps to analyze a combinational circuit
 - make sure that it is combinational and not sequential
 - No feedback path
 - No memory elements
 - derive its Boolean functions or truth table
 - verification by HDL
 - a verbal explanation of its function

4.3 Analysis of Combinational Circuit (2/5)

- A straight-forward procedure

- 3 inputs: A, B, C
- 2 outputs: F_1, F_2

$$F_2 = AB + AC + BC$$

$$T_1 = A + B + C$$

$$T_2 = ABC$$

$$T_3 = F_2' T_1$$

$$F_1 = T_3 + T_2$$

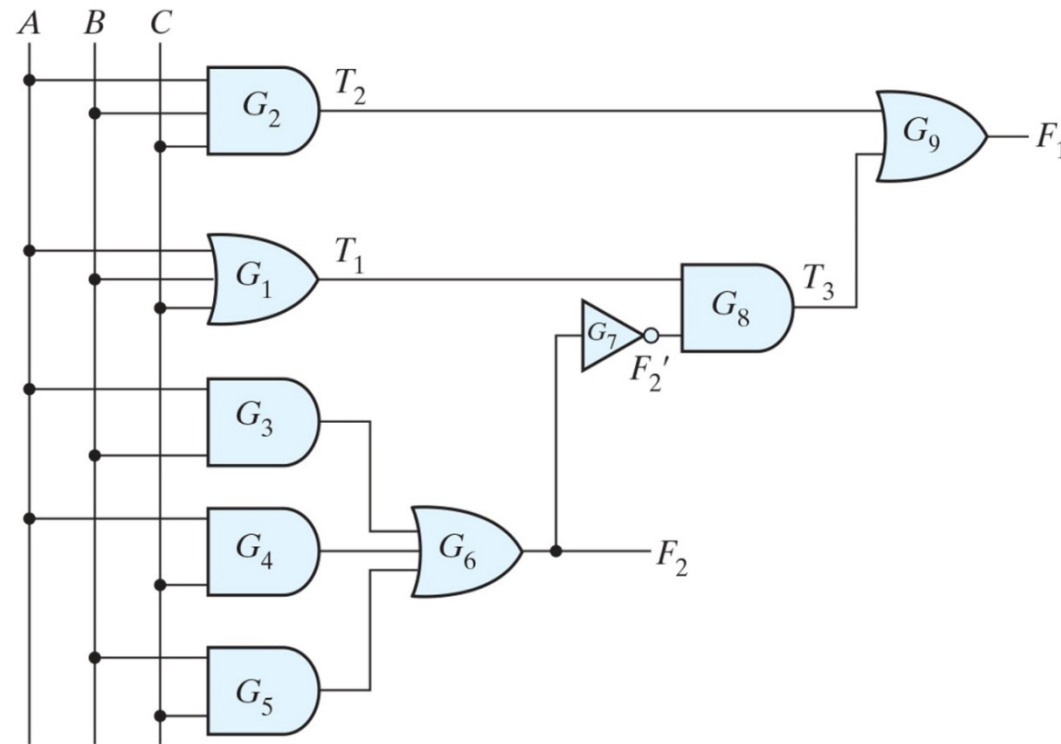


FIGURE 4.2

Logic diagram for analysis example.

4.3 Analysis of Combinational Circuit (3/5)

- $F_1 = T_3 + T_2 = F_2' T_1 + ABC$

$$= (AB + AC + BC)'(A + B + C) + ABC$$

$$= (A' + B')(A' + C')(B' + C')(A + B + C) + ABC$$

$$= (A' + B'C')(AB' + AC' + BC' + B'C) + ABC$$

$$= A'BC' + A'B'C + AB'C' + ABC$$
- A full-adder
 - F_1 : the sum
 - F_2 : the carry

4.3 Analysis of Combinational Circuit (4/5)

- Derive the truth table without knowing the Boolean functions

Table 4.1

Truth Table for the Logic Diagram of Fig. 4.2

A	B	C	F₂	F'₂	T₁	T₂	T₃	F₁
0	0	0	0	1	0	0	0	0
0	0	1	0	1	1	0	1	1
0	1	0	0	1	1	0	1	1
0	1	1	1	0	1	0	0	0
1	0	0	0	1	1	0	1	1
1	0	1	1	0	1	0	0	0
1	1	0	1	0	1	0	0	0
1	1	1	1	0	1	1	0	1

4.3 Analysis of Combinational Circuit (5/5)

- **Practice Exercise 4.1** Analyze the logic diagram in Fig. PE4.1 and find the Boolean expression for F_1 and F_2 .

Answer

$$T_1 = A + B + C$$

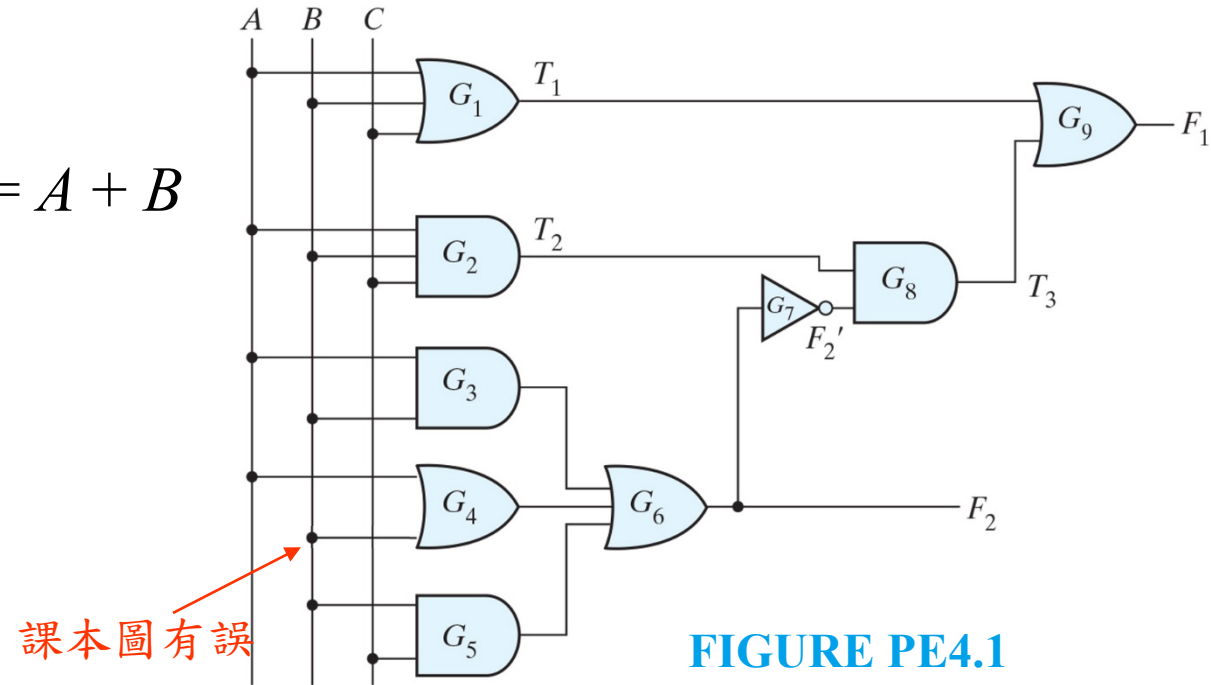
$$T_2 = ABC$$

$$F_2 = AB + (A + B) + BC = A + B + BC = A + B$$

$$F_2' = A'B'$$

$$T_3 = T_2 F_2' = (ABC)(A'B') = 0$$

$$F_1 = T_1 + T_3 = T_1 = A + B + C$$



4.4 Design Procedure (1/5)

- The design procedure of combinational circuits
 - State the problem (system spec.)
 - Determine the inputs and outputs
 - Assign symbols to the input and output variables
 - Derive the truth table
 - Derive the simplified Boolean functions
 - Draw the logic diagram and verify the correctness

4.4 Design Procedure (2/5)

- Functional description
 - Boolean function
 - HDL (Hardware description language)
 - Verilog HDL
 - VHDL
 - Schematic entry
- Logic minimization
 - number of gates
 - number of inputs to a gate
 - propagation delay
 - number of interconnection
 - limitations of the driving capabilities

4.4 Design Procedure (3/5)

Code Conversion Example: BCD to Excess-3 Code

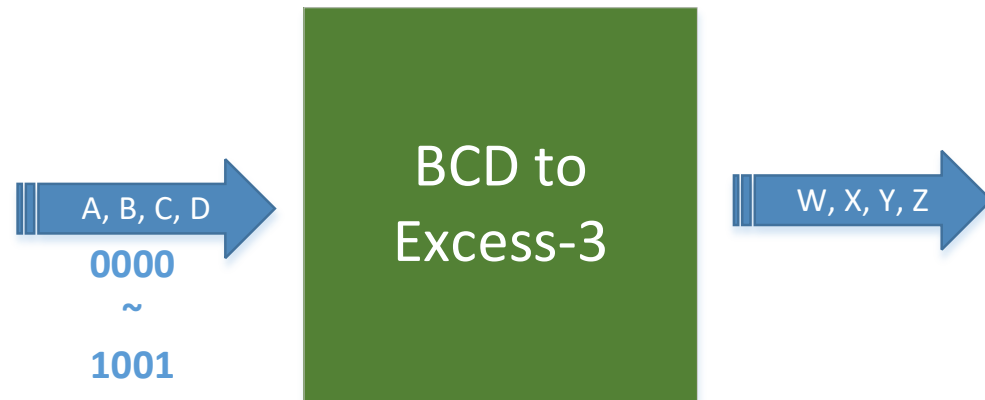


Table 4.2

Truth Table for Code Conversion Example

Input BCD				Output Excess-3 Code			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

4.4 Design Procedure (4/5)

Code Conversion Example: BCD to Excess-3 Code

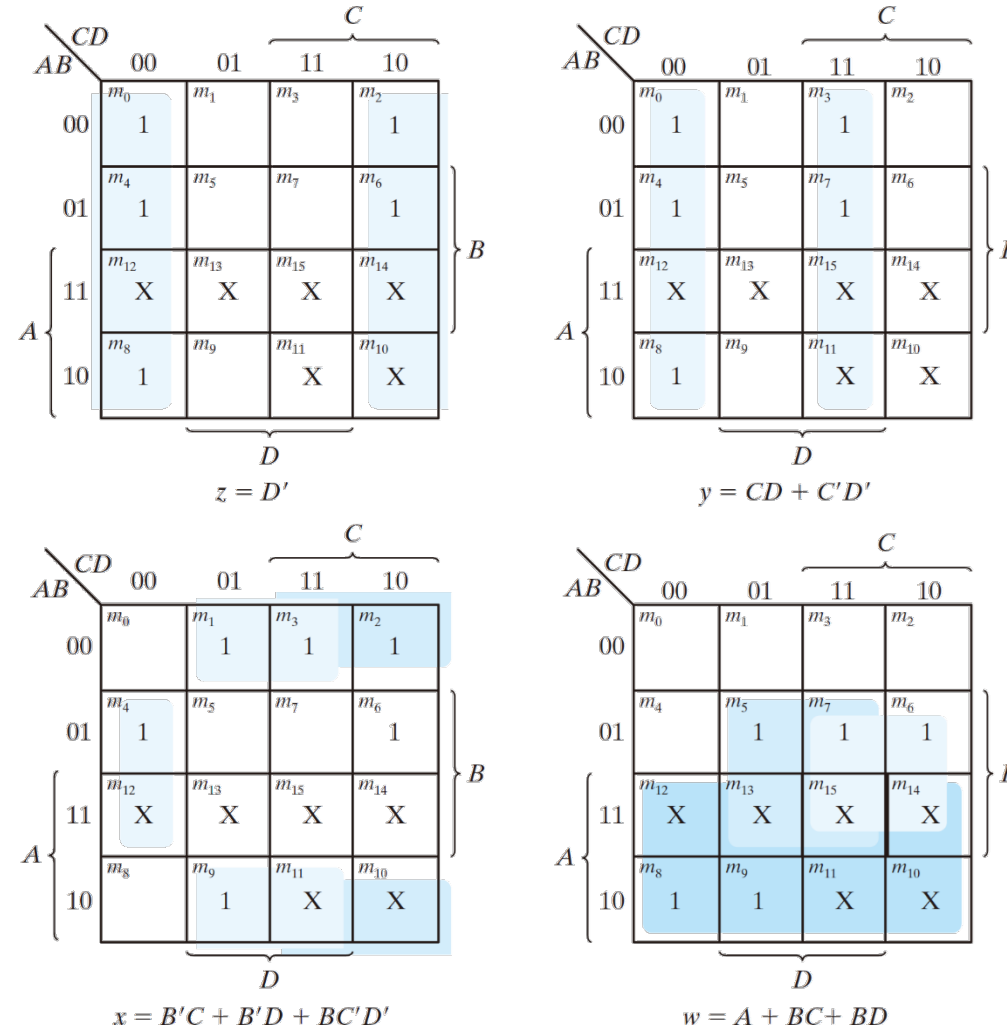


FIGURE 4.3

Maps for BCD-to-excess-3 code converter

Chih-Cheng Tseng, CSE of NTU

4.4 Design Procedure (5/5)

Code Conversion Example: BCD to Excess-3 Code

- The simplified Boolean functions

- $z = D'$
- $y = CD + C'D'$
- $x = B'C + B'D + BC'D'$
- $w = A + BC + BD$

- Another implementation

- $z = D'$
- $y = CD + C'D' = CD + (C + D)'$
- $x = B'C + B'D + BC'D' = B'(C + D) + B(C + D)'$
- $w = A + BC + BD = A + B(C + D)$

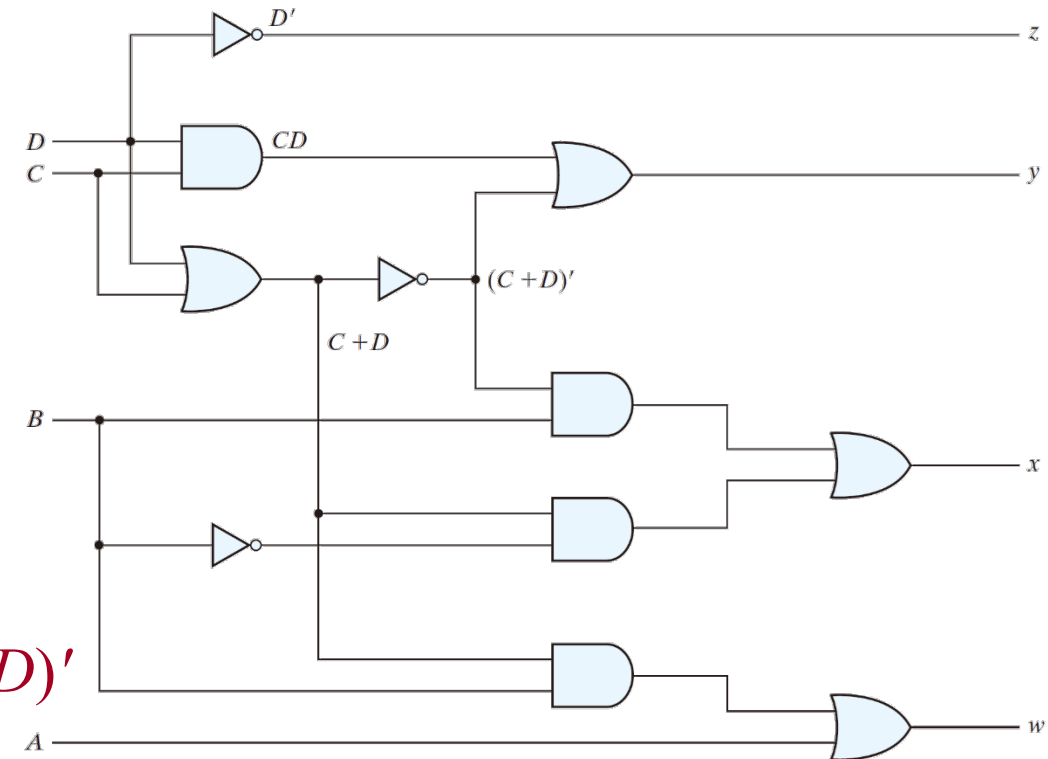


FIGURE 4.4
Logic diagram for BCD-to-excess-3 code converter

4.5 Binary Adder-Subtractor (1/17)

Half-Adder (HA)

- Four basic operations
 - $0 + 0 = 00$
 - $0 + 1 = 01$
 - $1 + 0 = 01$
 - $1 + 1 = 10$
- Two input variables
 - x
 - y
- Two output variables
 - C : carry
 - S : sum

- The truth table

Table 4.3
Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

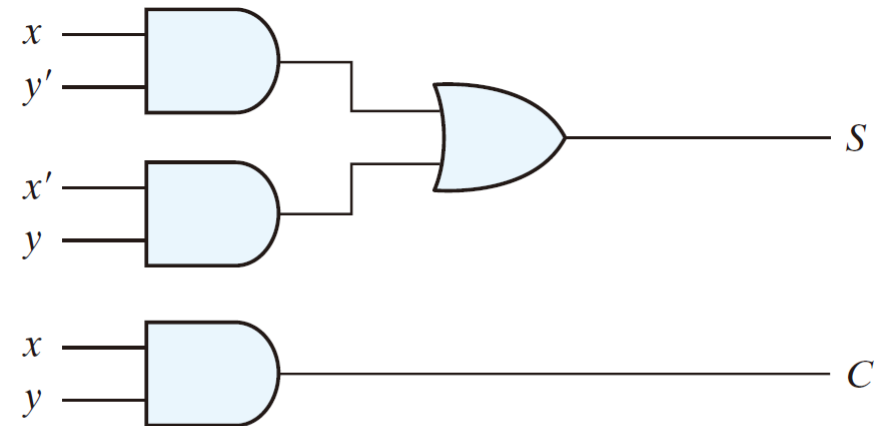
- Boolean expressions for an HA
 - $S = x'y + xy'$
 - $C = xy$

4.5 Binary Adder-Subtractor (2/17)

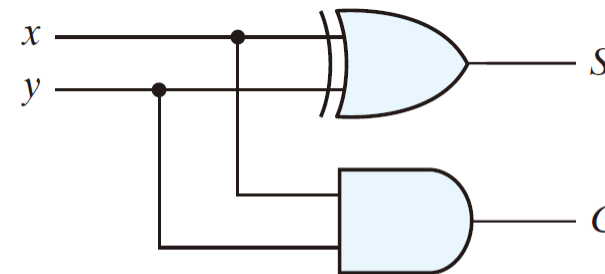
Half-Adder (HA)

- The flexibility for implementation

- $S = x'y + xy'$
- $S = x \oplus y$
- $S = (x + y)(x' + y')$
- $S' = xy + x'y' \rightarrow S = (C + x'y')'$
- $C = xy = (x' + y')'$



(a) $S = xy' + x'y$
 $C = xy$



(b) $S = x \oplus y$
 $C = xy$

FIGURE 4.5
Implementation of half adder

4.5 Binary Adder-Subtractor (3/17)

Full Adder (FA)

- The arithmetic sum of three input bits
 - x, y : two significant bits
 - z : the carry bit from the previous lower significant bit
- Two output bits
 - C : carry
 - S : sum

4.5 Binary Adder-Subtractor (4/17)

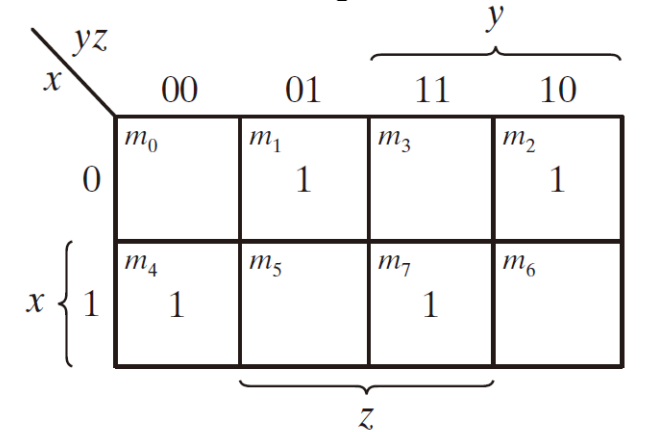
Full Adder (FA)

- The truth table

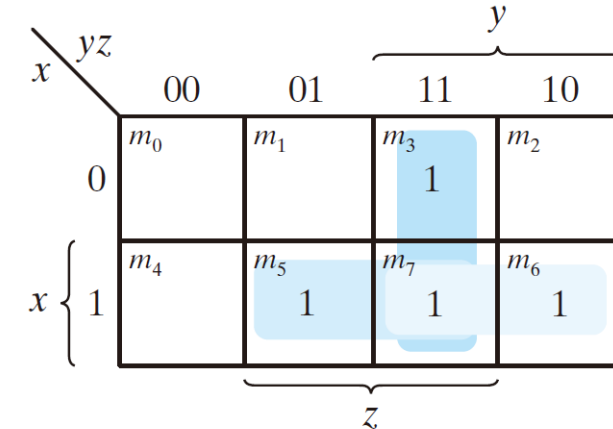
Table 4.4
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- Boolean expressions of an FA



$$(a) S = x'y'z + x'yz' + xy'z' + xyz$$



$$(b) C = xy + xz + yz$$

FIGURE 4.6
K-Maps for full adder

4.5 Binary Adder-Subtractor (5/17)

Full Adder (FA)

- Implementation of an FA

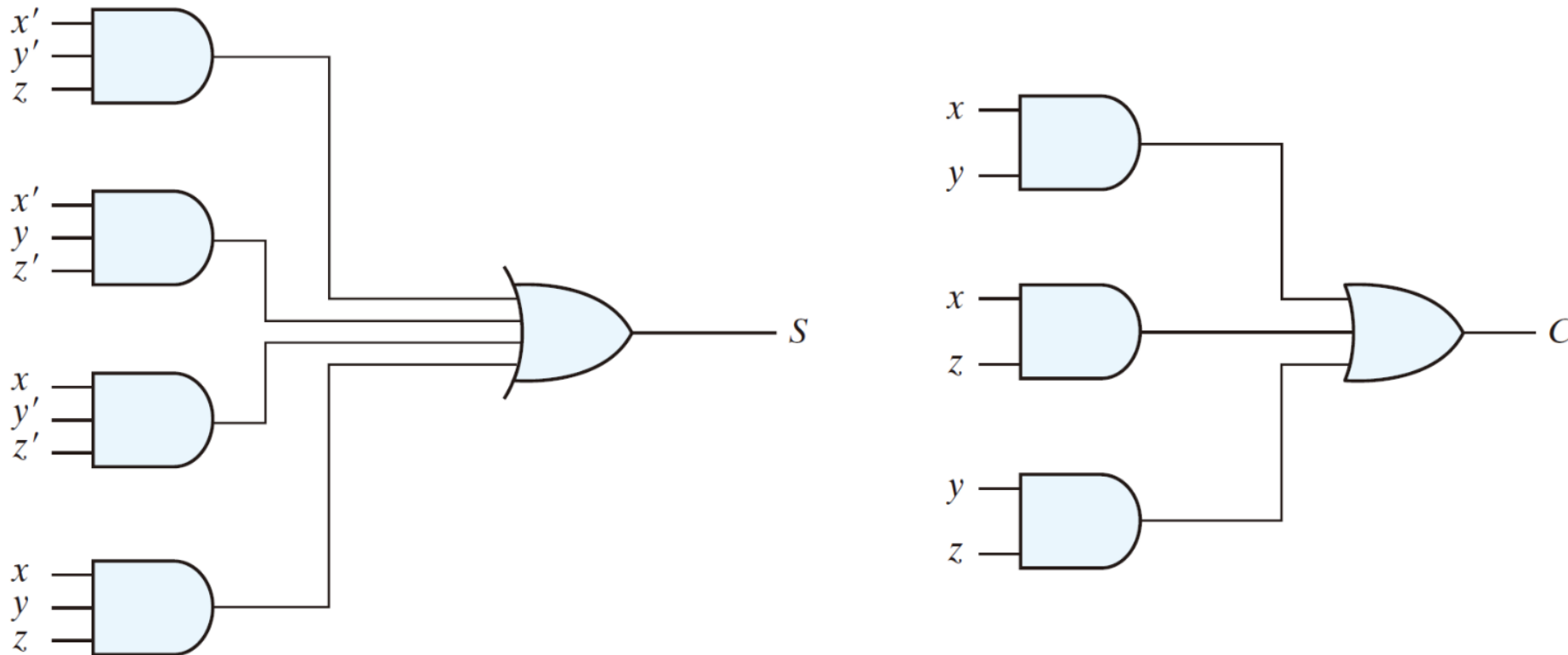


FIGURE 4.7

Implementation of full adder in sum-of-products form

4.5 Binary Adder-Subtractor (6/17)

- **Practice Exercise 4.2** Explain how a half adder and a full adder differ in their functionality.

Answer: A half adder adds only two (data) bits to produce a sum and carry-out bit. A full adder adds input bits (two data bits and a carry-in bit) to produce a sum and carry-out bit.

4.5 Binary Adder-Subtractor (7/17)

- Implement an FA with two HAs and an OR gate
 - $S = xy'z' + x'yz' + xyz + x'y'z = z'(xy' + x'y) + z(xy + x'y')$
 $= z'(xy' + x'y) + z(xy' + x'y)' = z \oplus (x \oplus y)$
 - $C = xy'z + x'yz + xy = (xy' + x'y)z + xy = (x \oplus y)z + xy$

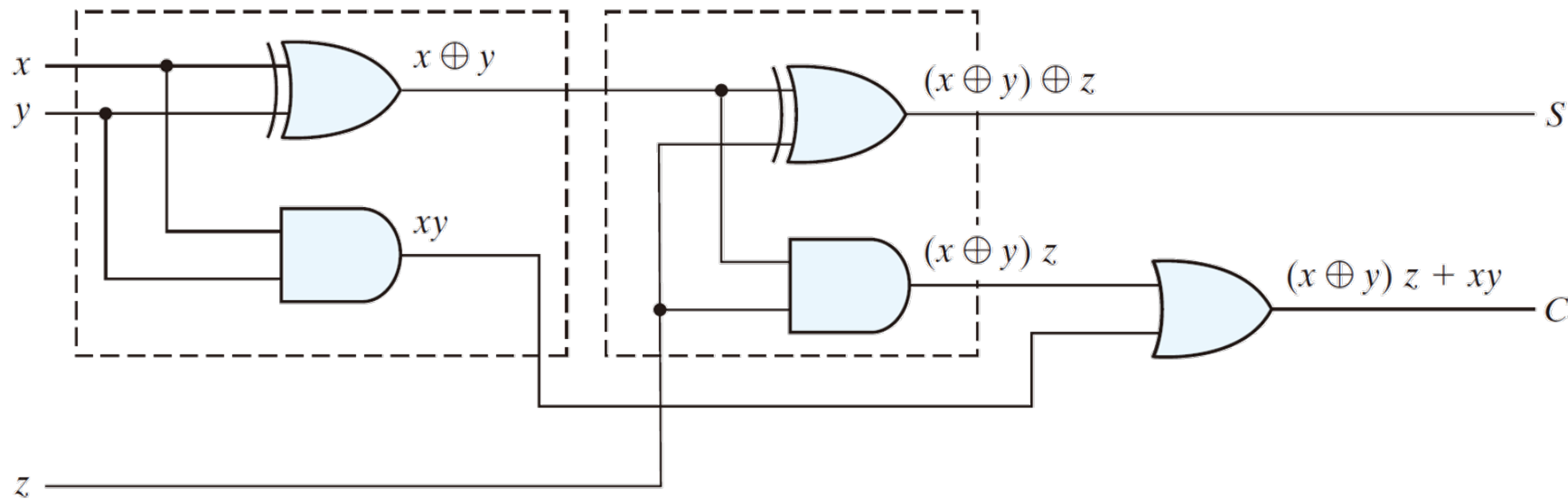


FIGURE 4.8

Implementation of full adder with two half adders and an OR gate

4.5 Binary Adder-Subtractor (8/17)

Binary Adder

- Using FAs to construct a 4-bit ripple adder

- Inputs

- Augend (被加數): (A_3, A_2, A_1, A_0)
- Addend (加數): (B_3, B_2, B_1, B_0)
- One carry-in bit: C_0

- Outputs

- Sum: (S_3, S_2, S_1, S_0)
- One carry-out bit: C_4

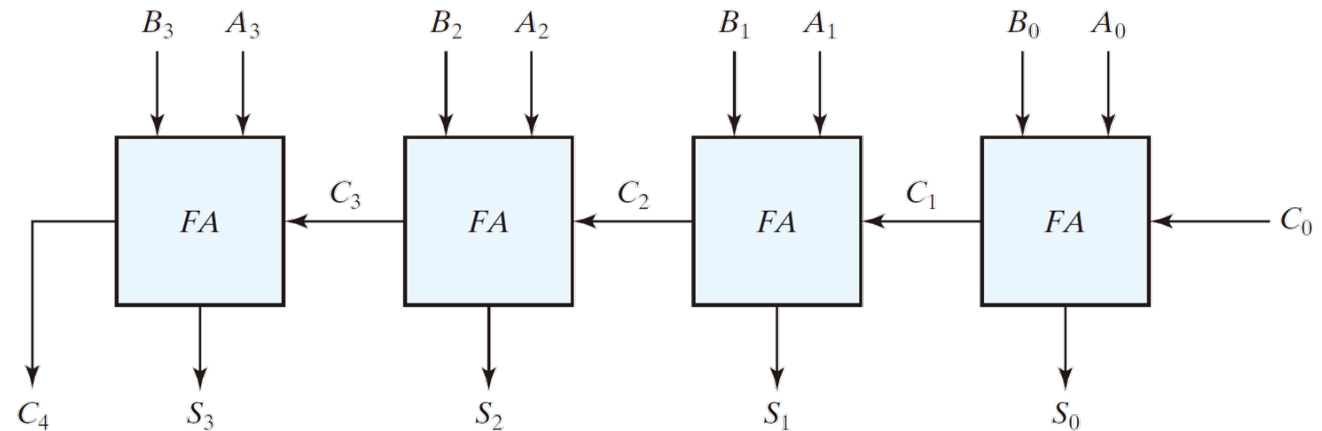


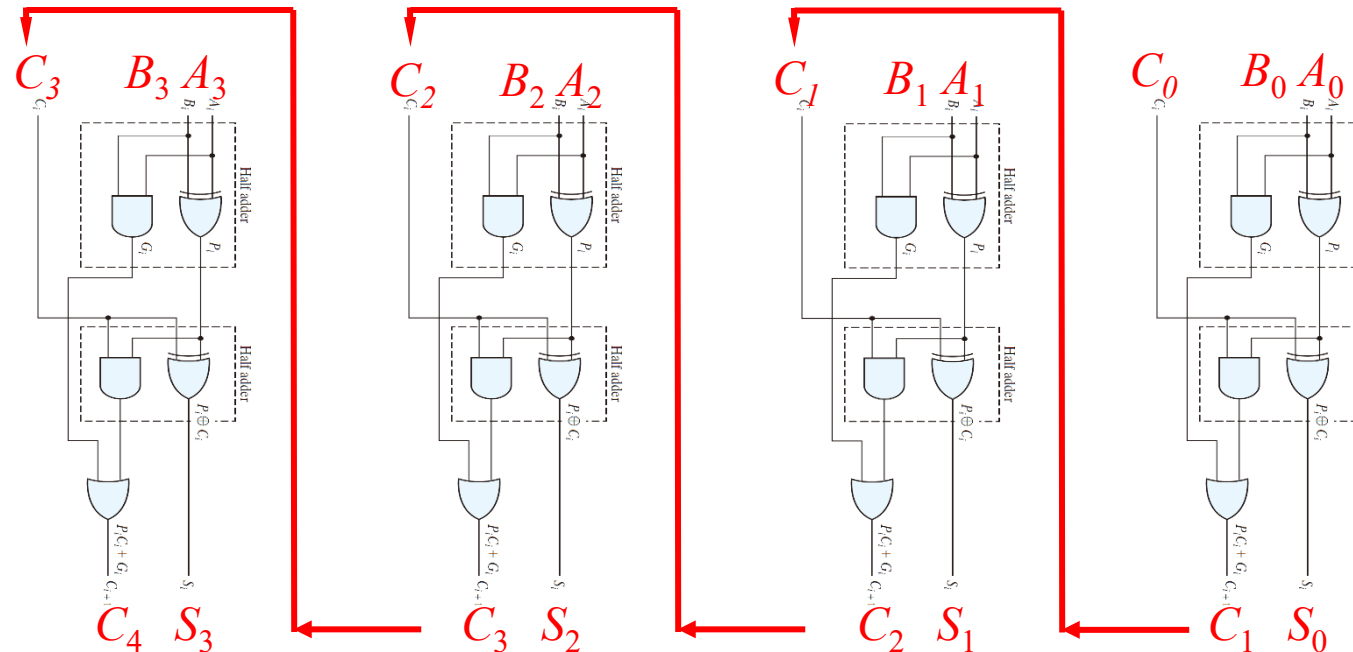
FIGURE 4.9
Four-bit adder

Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

4.5 Binary Adder-Subtractor (9/17)

Binary Adder

- Carry propagation
 - $(A_0, B_0, C_0) \rightarrow C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow (C_4, S_3)$
 - In the worst case, for C_i to C_{i+1} propagates through an AND gate and an OR gate.
 - For n -bit adder, there are $2n$ gate levels for the carry to propagate from input to output.



4.5 Binary Adder-Subtractor (10/17)

Binary Adder

- **Practice Exercise 4.3** What is the main drawback of a ripple adder?
Answer: The time required to add long data words may be prohibitive, because the carry has to propagate from the least significant bit to the most significant bit.

4.5 Binary Adder-Subtractor (11/17)

Binary Adder

- Reduce the carry propagation delay
 - employ faster gates
 - **look-ahead carry** (前瞻進位): more complex mechanism, yet faster

- Carry lookahead logic

- carry propagate: $P_i = A_i \oplus B_i$
- carry generate: $G_i = A_i B_i$
- sum: $S_i = P_i \oplus C_i$
- carry: $C_{i+1} = G_i + P_i C_i$
 - $C_1 = G_0 + P_0 C_0$
 - $C_2 = G_1 + P_1 C_1 = G_1 + P_1(G_0 + P_0 C_0)$
 $= G_1 + P_1 G_0 + P_1 P_0 C_0$
 - $C_3 = G_2 + P_2 C_2 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

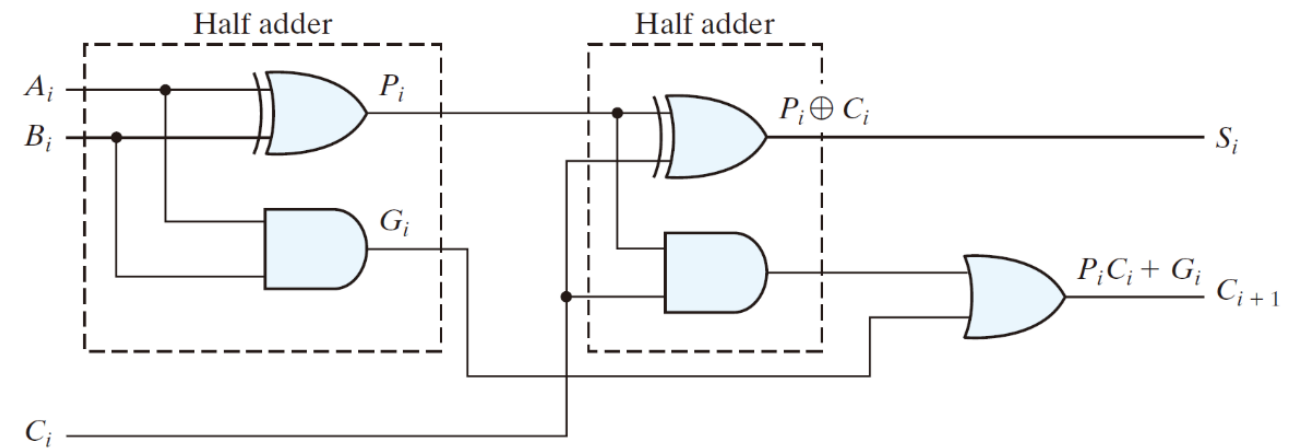


FIGURE 4.10
Full adder with P and G shown

4.5 Binary Adder-Subtractor (12/17)

Binary Adder

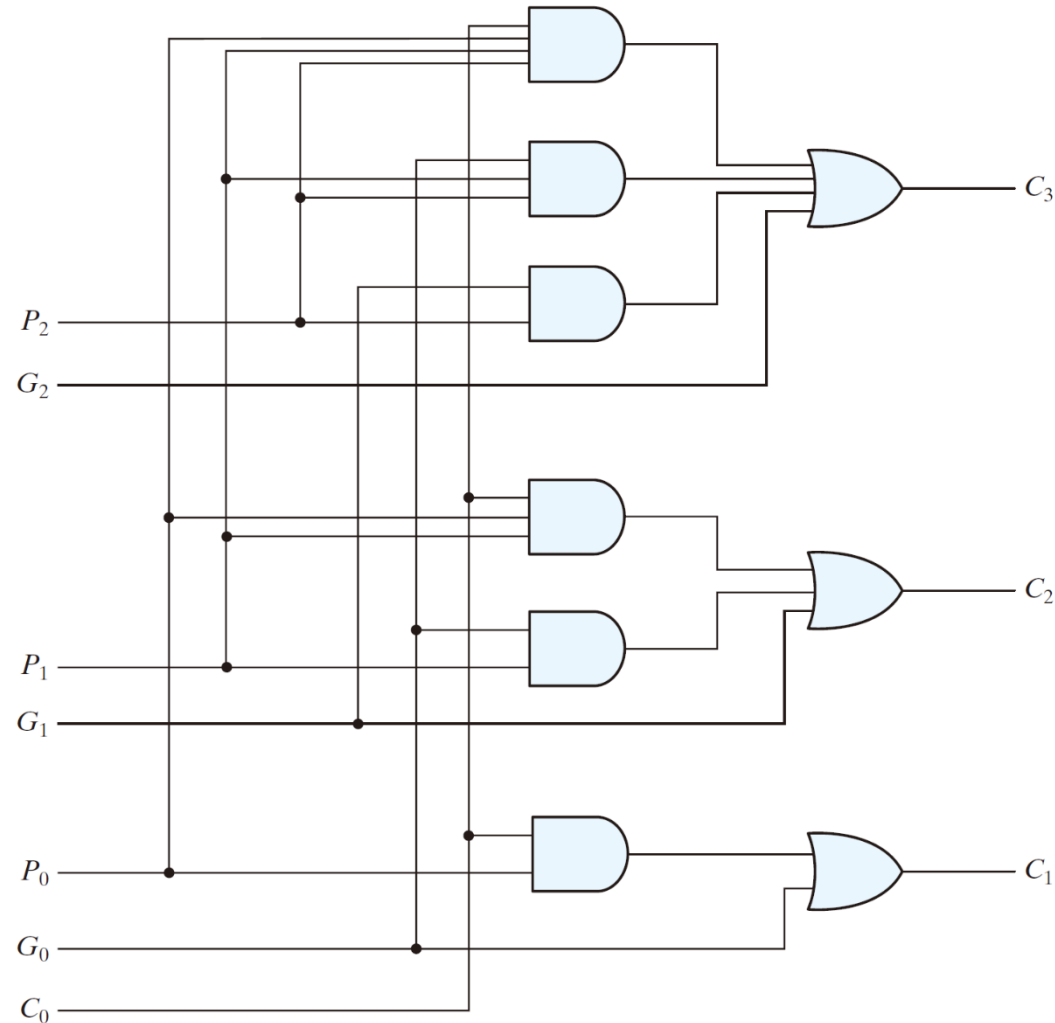


FIGURE 4.11

Logic diagram of carry lookahead generator

Chih-Cheng Tseng, CSE of NTOU

4.5 Binary Adder-Subtractor (13/17)

Binary Adder

- **Practice Exercise 4.4** What is the main drawback of a lookahead adder?
Answer: Its hardware is more complex than that of a ripple carry adder.
- **Practice Exercise 4.5** Adding the following two binary words and find the sum and carry bit: $A = 1100_0101$, $B = 1010_1010$.
Answer: Sum = 0110_1111 , Carry = 1.

4.5 Binary Adder-Subtractor (14/17)

Binary Adder

- 4-bit carry-look ahead adder
 - propagation delay

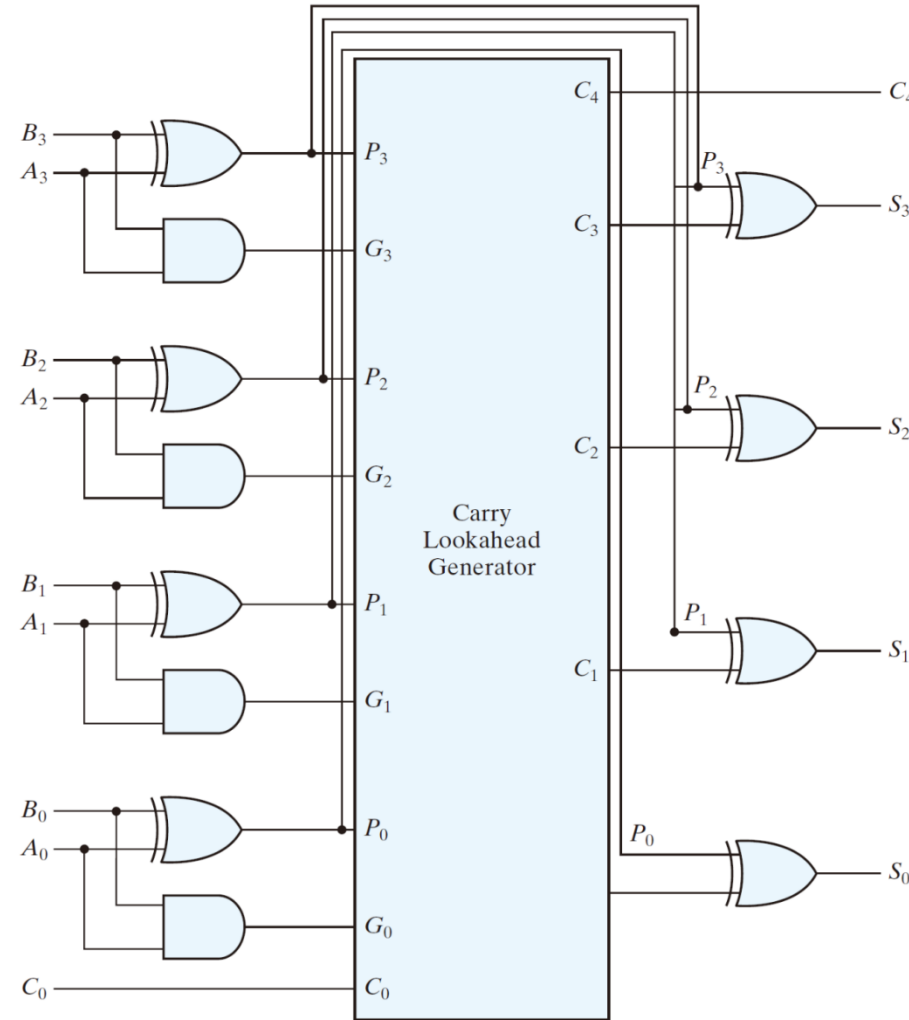


FIGURE 4.12

Four-bit adder with carry lookahead

Chih-Cheng Tseng, CSE of NTU

4.5 Binary Adder-Subtractor (15/17)

Binary Adder/Subtractor

- $A - B = A + (2\text{'s complement of } B)$
 - $M = 0, B \oplus 0 = B \rightarrow A + B$
 - $M = 1, B \oplus 1 = B' \rightarrow A + B' + 1$

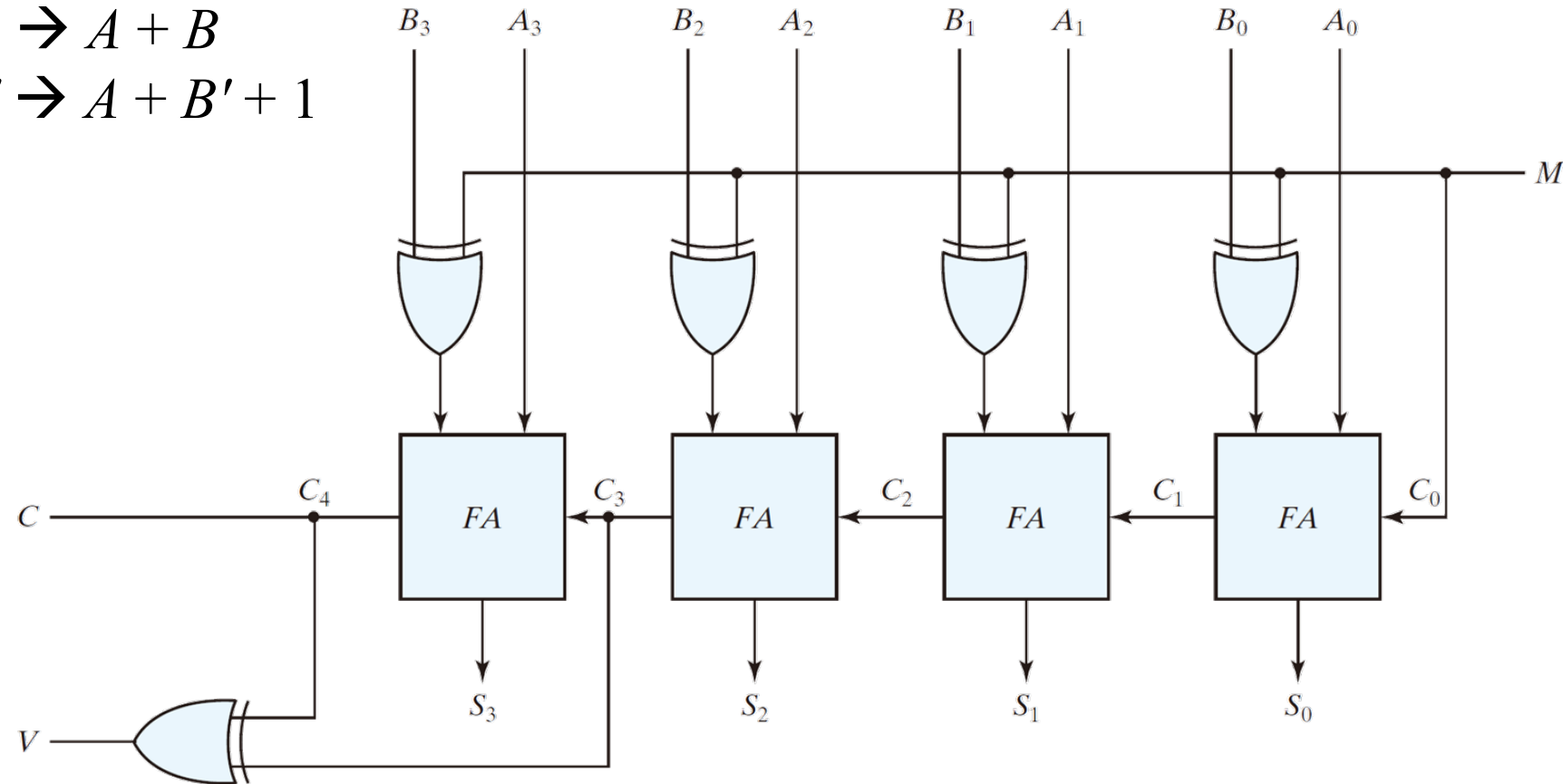


FIGURE 4.13

Four-bit adder-subtractor (with overflow detection)

4.5 Binary Adder-Subtractor (16/17)

Binary Adder/Subtractor

- **Practice Exercise 4.6** Find $A - B$ with $A = 1001$ and $B = 0110$
Answer: $A - B = 1_0011$

4.5 Binary Adder-Subtractor (17/17)

Binary Subtractor

- Overflow
 - The storage is limited
 - May occur when two numbers added are both positive or negative
 - $V = C_{i+1} \oplus C_i$
 - $V = 0$, no overflow; $V = 1$, overflow
- Example: An 8-bit storage can represent $+127 \sim -128$. Perform $A + B$ if
 - (a) $A = +70_{10} = 0100_0110_2$ and $B = +80_{10} = 0101_0000_2$;
 - (b) $A = -70_{10} = 1011_1010_2$ and $B = -80_{10} = 1011_0000_2$

carries:	0 1
+70	0 1000110
+80	0 1010000
+150	1 0010110

carries:	1 0
-70	1 0111010
-80	1 0110000
-150	0 1101010

4.6 Decimal Adder (1/4)

- Add 2 BCD's
 - 9 inputs: 2 BCDs and 1 carry-in
 - 5 outputs: 1 BCD and 1 carry-out
- Design approaches
 - Theoretically, a truth table with 2^9 entries is needed
 - However,
 - the sum $\leq 9 + 9 + 1 = 19$
 - modifying the output of a 4-bit binary adder from binary to BCD

4.6 Decimal Adder (2/4)

- The truth table

Table 4.5
Derivation of BCD Adder

Binary Sum					BCD Sum					Decimal
<i>K</i>	<i>Z</i> ₈	<i>Z</i> ₄	<i>Z</i> ₂	<i>Z</i> ₁	<i>C</i>	<i>S</i> ₈	<i>S</i> ₄	<i>S</i> ₂	<i>S</i> ₁	
0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	2
0	0	0	1	1	0	0	0	1	1	3
0	0	1	0	0	0	0	1	0	0	4
0	0	1	0	1	0	0	1	0	1	5
0	0	1	1	0	0	0	1	1	0	6
0	0	1	1	1	0	0	1	1	1	7
0	1	0	0	0	0	1	0	0	0	8
0	1	0	0	1	0	1	0	0	1	9
0	1	0	1	0	1	0	0	0	0	10
0	1	0	1	1	1	0	0	0	1	11
0	1	1	0	0	1	0	0	1	0	12
0	1	1	0	1	1	0	0	1	1	13
0	1	1	1	0	1	0	1	0	0	14
0	1	1	1	1	1	0	1	0	1	15
1	0	0	0	0	1	0	1	1	0	16
1	0	0	0	1	1	0	1	1	1	17
1	0	0	1	0	1	1	0	0	0	18
1	0	0	1	1	1	1	0	0	1	19

4.6 Decimal Adder (3/4)

- Modifications are needed if the sum > 9
 - $C = 1$
 - $K = 1$
 - $Z_8Z_4 = 1$
 - $Z_8Z_2 = 1$
 - modification: $-(10)_d$ or $+6$
- $C = K + Z_8Z_4 + Z_8Z_2$

4.6 Decimal Adder (4/4)

- Block diagram

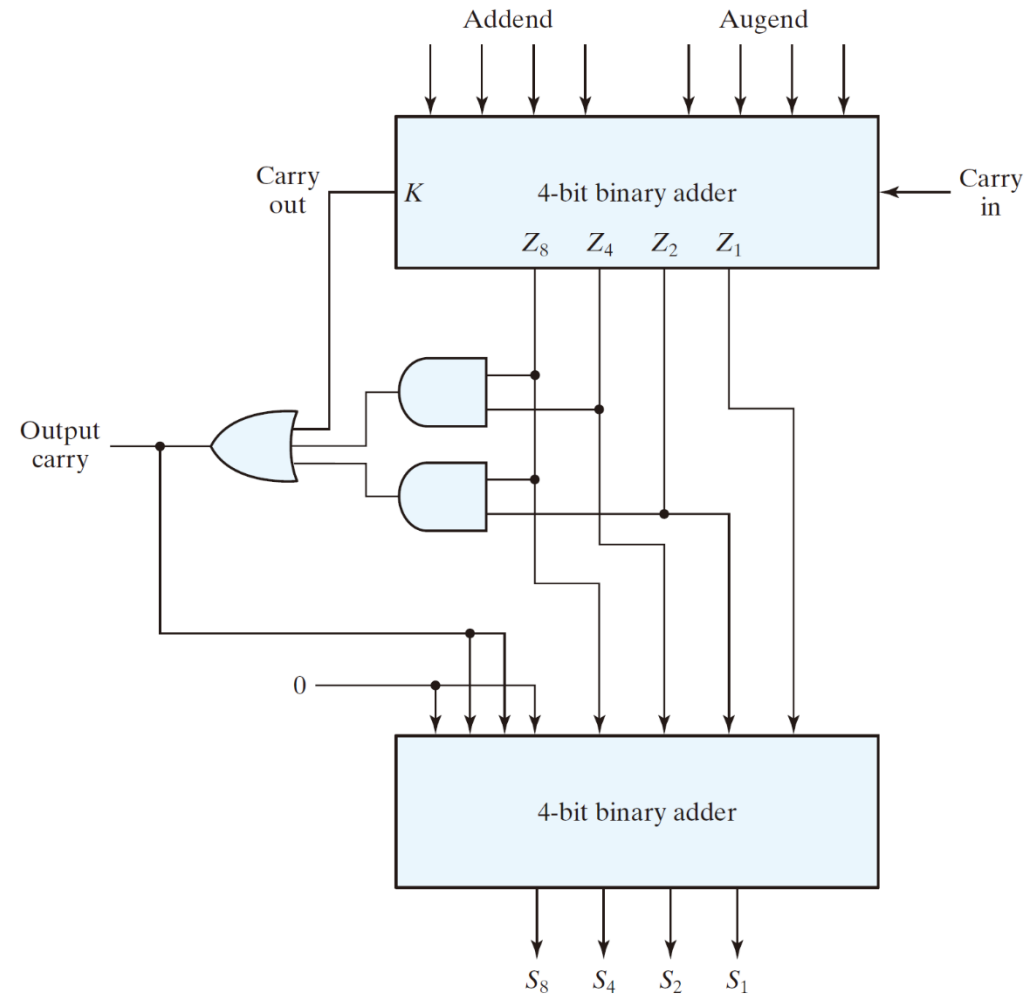


FIGURE 4.14
Block diagram of a BCD adder

4.7 Binary Multiplier (1/3)

- Partial products (部分乘積): AND operations
- 2-bit by 2-bit binary multiplier

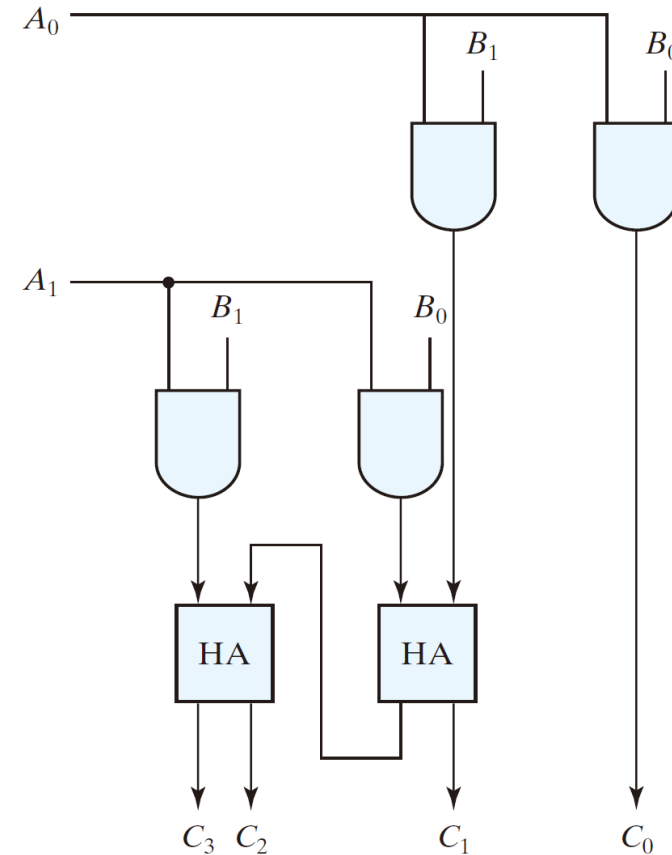
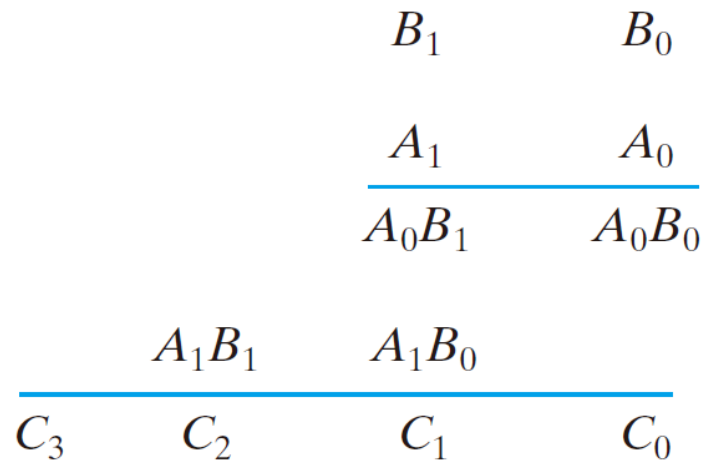


FIGURE 4.15
Two-bit by two-bit binary multiplier

4.7 Binary Multiplier (2/3)

- 4-bit ($B_3B_2B_1B_0$) by 3-bit ($A_2A_1A_0$) binary multiplier

$$\begin{array}{r}
 B_3B_2B_1B_0 \\
 \times \quad A_2A_1A_0 \\
 \hline
 P_3P_2P_1P_0 \\
 Q_3Q_2Q_1Q_0 \\
 R_3R_2R_1R_0 \\
 \hline
 C_6C_5C_4C_3C_2C_1C_0
 \end{array}$$

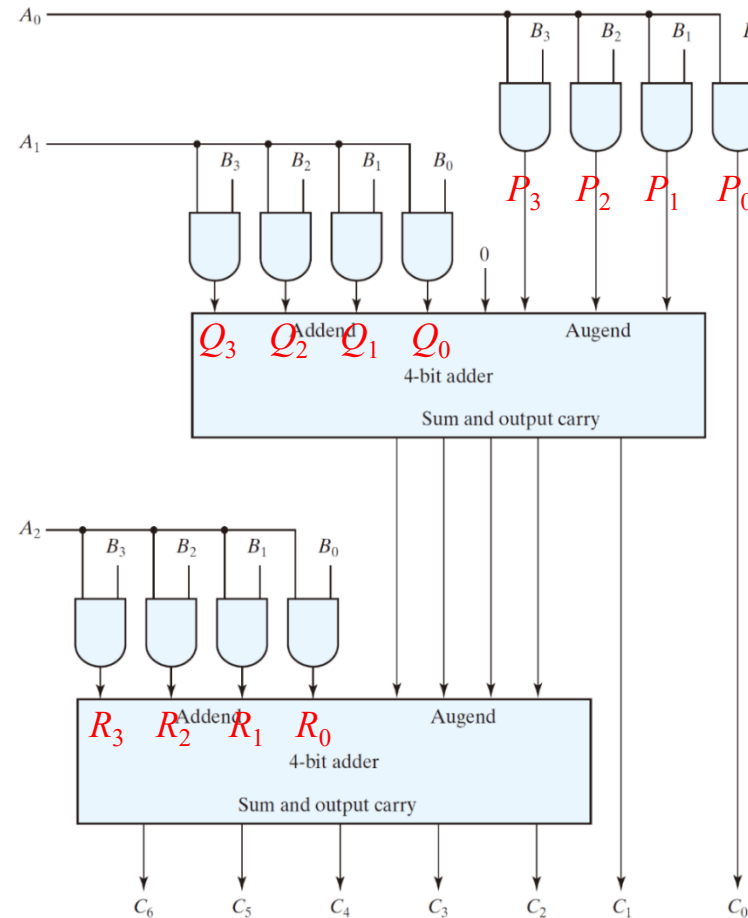


FIGURE 4.16
Four-bit by three-bit binary multiplier

4.7 Binary Multiplier (3/3)

- **Practice Exercise 4.7** Find the product $(0101)_2 \times (1001)_2$

Answer: 0101101_2

4.8 Magnitude Comparator (1/3)

- The comparison of two n -bit binaries
 - outputs: $A > B$, $A = B$, $A < B$
- Design approaches
 - The truth table
 - 2^{2n} entries - too cumbersome for large n
 - Use the inherent regularity of the problem
 - reduce design efforts
 - reduce human errors

4.8 Magnitude Comparator (2/3)

- Based on an algorithm to design combinational logic
 - $A = A_3A_2A_1A_0; B = B_3B_2B_1B_0$
 - $A = B$ if $A_3 = B_3, A_2 = B_2, A_1 = B_1$, and $A_0 = B_0$
 - equality: $x_i = A_iB_i + A_i'B_i'$, for $i = 0, 1, 2, 3$
 - $(A = B) = x_3x_2x_1x_0$
 - $(A > B) = A_3B_3' + x_3A_2B_2' + x_3x_2A_1B_1' + x_3x_2x_1A_0B_0'$
 - $(A < B) = A_3'B_3 + x_3A_2'B_2 + x_3x_2A_1'B_1 + x_3x_2x_1A_0'B_0$
- Implementation
 - $x_i = (A_iB_i' + A_i'B_i)' = (A_i \oplus B_i)'$

4.8 Magnitude Comparator (3/3)

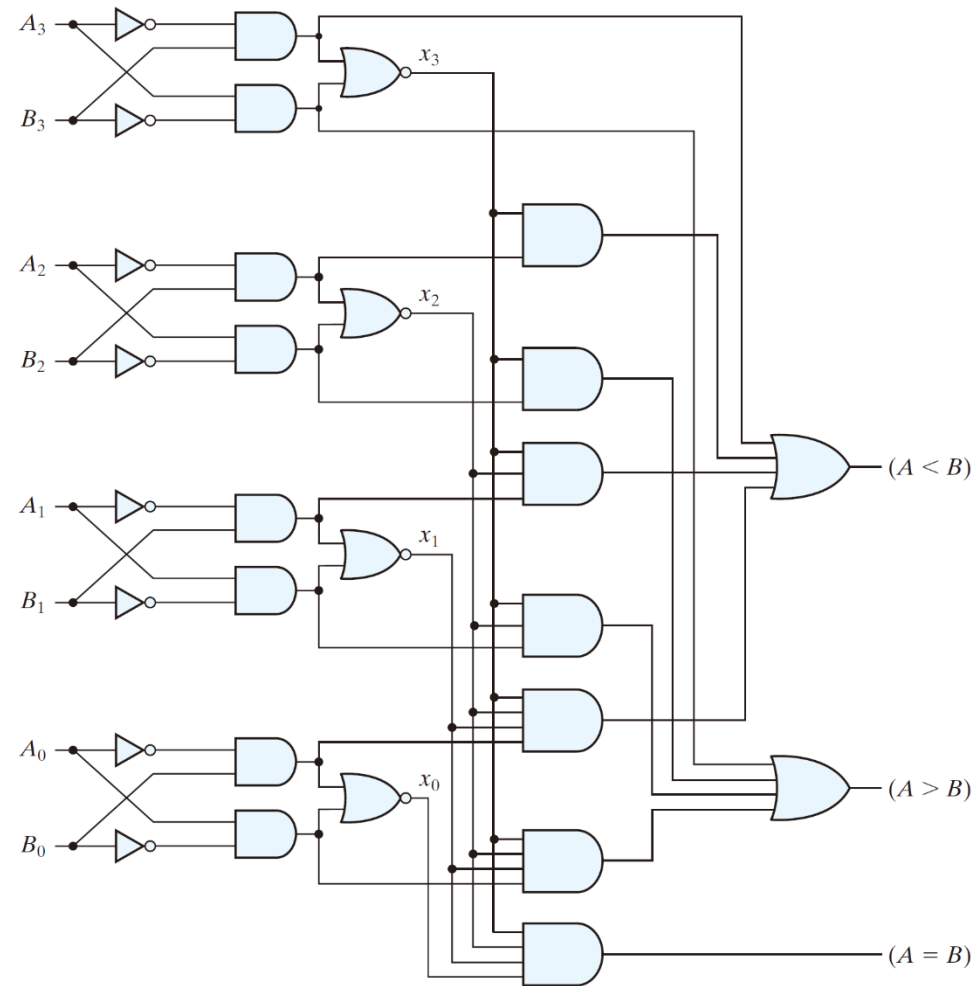


FIGURE 4.17
Four-bit magnitude comparator

4.9 Decoders (1/8)

- A n -to- 2^n decoder
 - A binary code of n bits $\rightarrow 2^n$ distinct information
 - n input variables
 - Among the 2^n output lines, only one output can be active (high) at any time

Table 4.6
Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

4.9 Decoders (2/8)

- An implementation

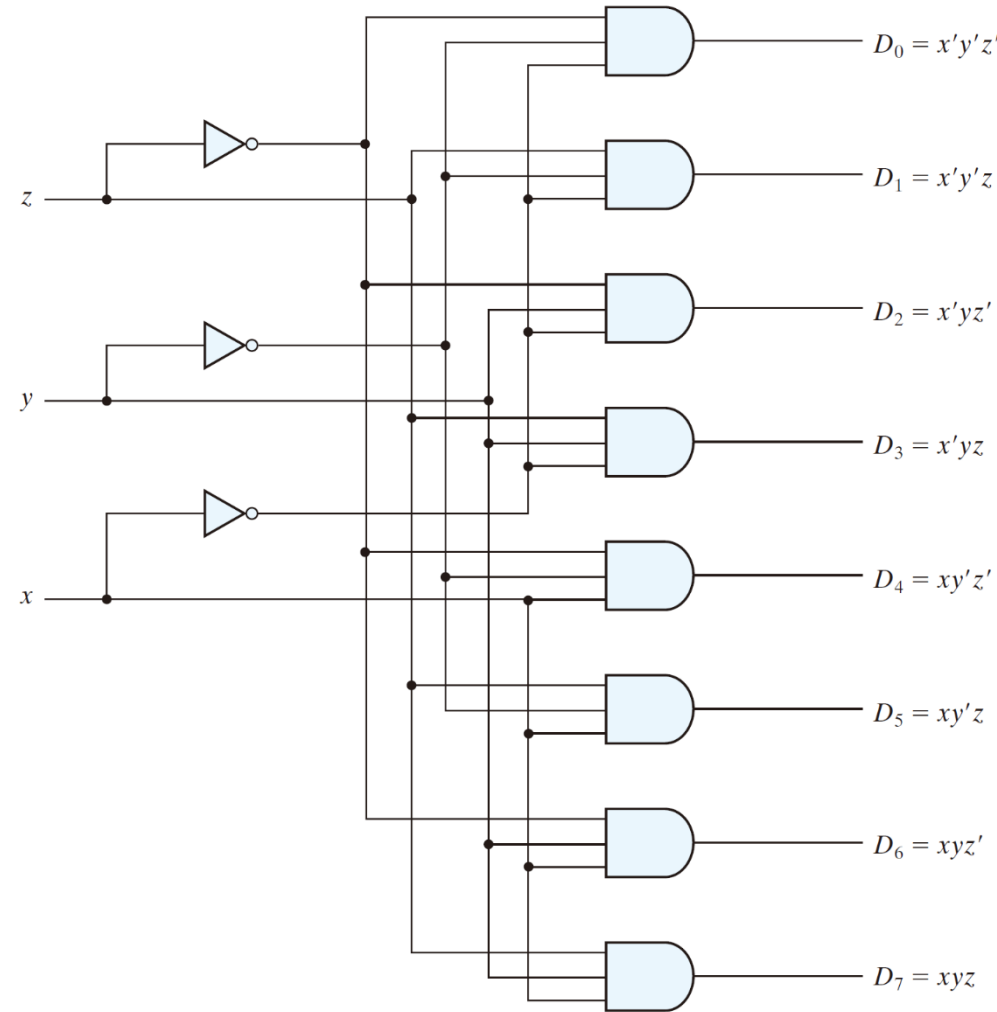


FIGURE 4.18
Three-to-eight-line decoder

4.9 Decoders (3/8)

- Each output of an n -to- 2^n decoder is a minterm
- A decoder and external OR gate(s) can be applied to implement any Boolean function of n input variables

4.9 Decoders (4/8)

- Demultiplexers (解多工器): 1 input, n selects, and 2^n outputs
 - A n -to- 2^n decoder with an enable (E) input
 - A circuit that receives information from a single line and directs it to one of 2^n possible output lines by using the n selects.

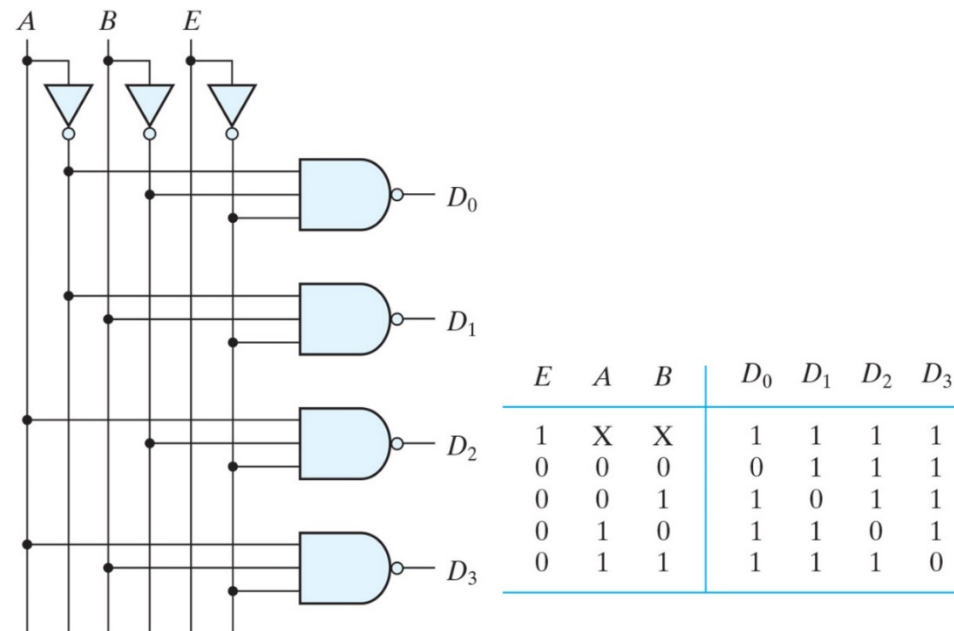


FIGURE 4.19

Two-to-four-line decoder with enable input.

4.9 Decoders (5/8)

- Expansion: two 3×8 decoders $\rightarrow$ one 4×16 decoder
- How to construct a 5×32 decoder in terms of 3×8 and 2×4 decoders?

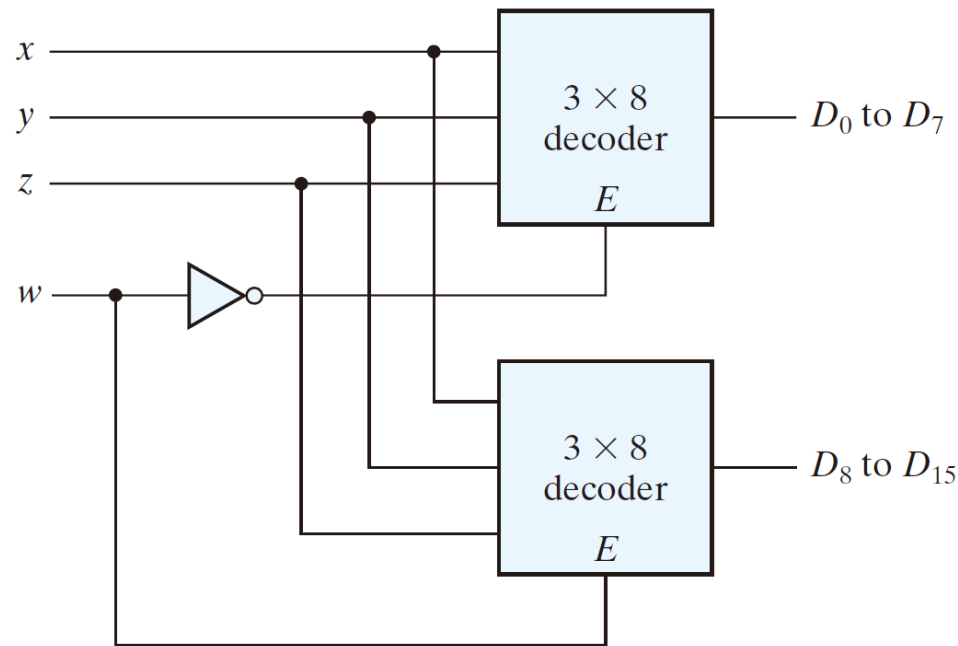


FIGURE 4.20

4×16 decoder constructed with two 3×8 decoders

4.9 Decoders (6/8)

- **Practice Exercise 4.8** Draw a logic diagram constructing a 3×8 decoder with active-low enable, using a pair of 2×4 decoders; also draw a truth table for the configuration.

Answer:

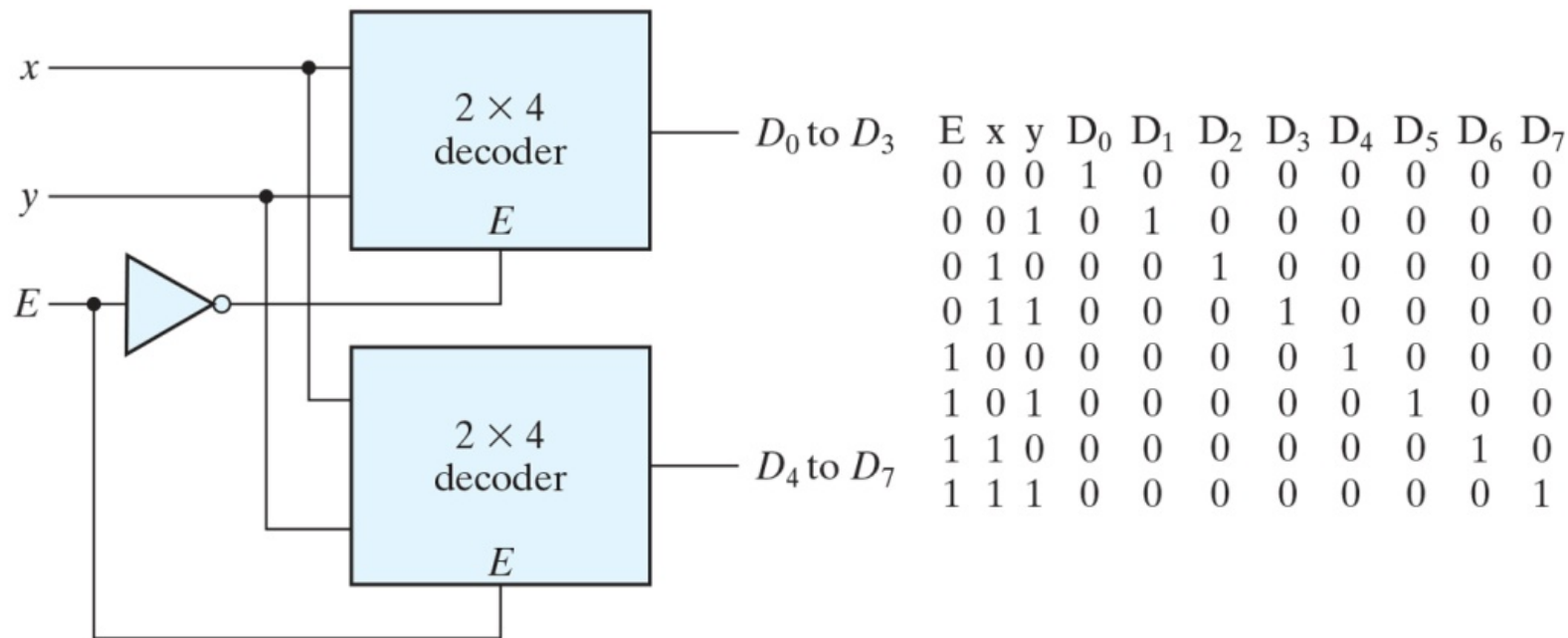


FIGURE PE4.8

4.9 Decoders (7/8)

- Apply 3×8 decoder to implement a 4-bit FA
 - $S(x, y, z) = \Sigma(1, 2, 4, 7)$
 - $C(x, y, z) = \Sigma(3, 5, 6, 7)$

Table 4.4
Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

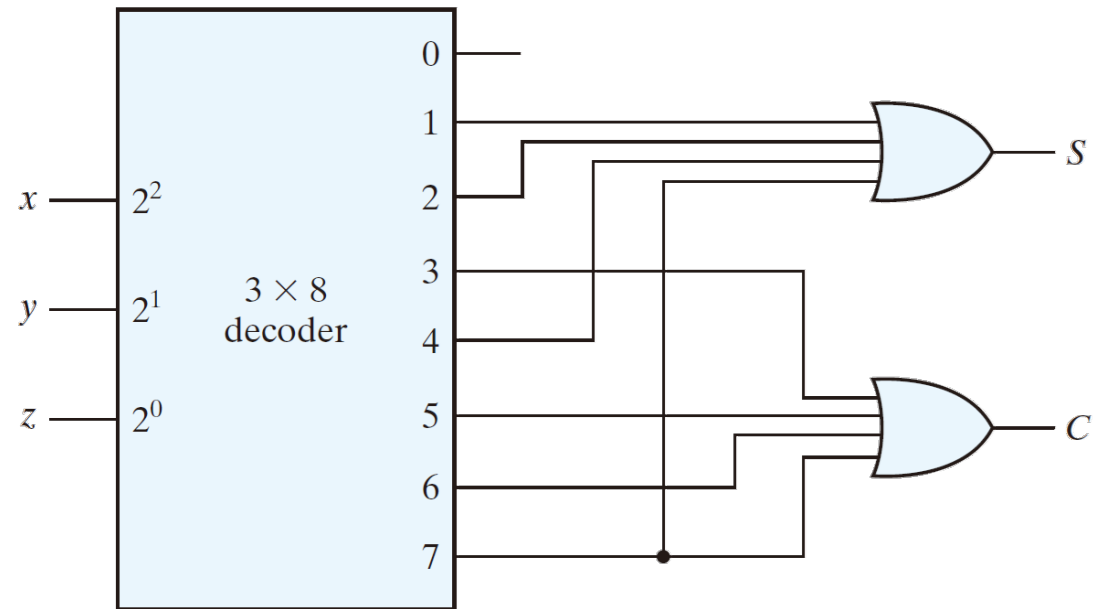


FIGURE 4.21

Implementation of a full adder with a decoder

4.9 Decoders (8/8)

- A function F having a list of k minterms
 - two possible approaches using a decoder
 - if $k < 2^n/2$, using OR (minterms of F) with k inputs,
 - Otherwise, using NOR (minterms of F') with $2^n - k$ inputs
- In general, this is not a practical implementation.

4.10 Encoders (1/4)

- The inverse function of a decoder
- The encoder can be implemented with three OR gates.

Table 4.7
Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

4.10 Encoders (2/4)

- Priority Encoder
 - Resolve the ambiguity of illegal inputs
 - Only one of the inputs is encoded
 - D_3 has the highest priority
 - D_0 has the lowest priority
 - X: don't-care conditions
 - V : valid output indicator

Table 4.8
Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

4.10 Encoders (3/4)

- The maps for simplifying outputs x and y

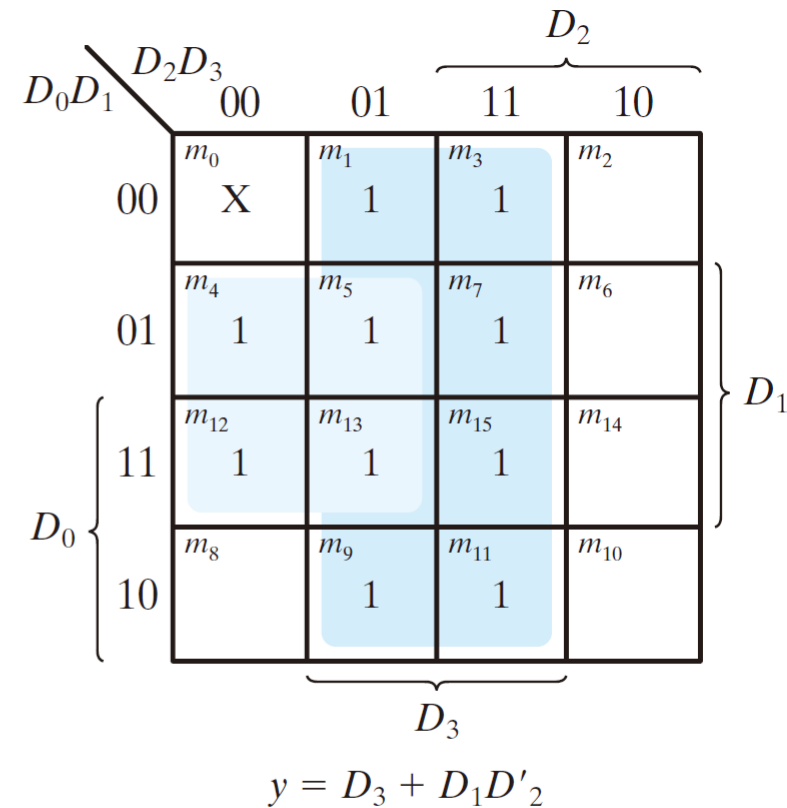
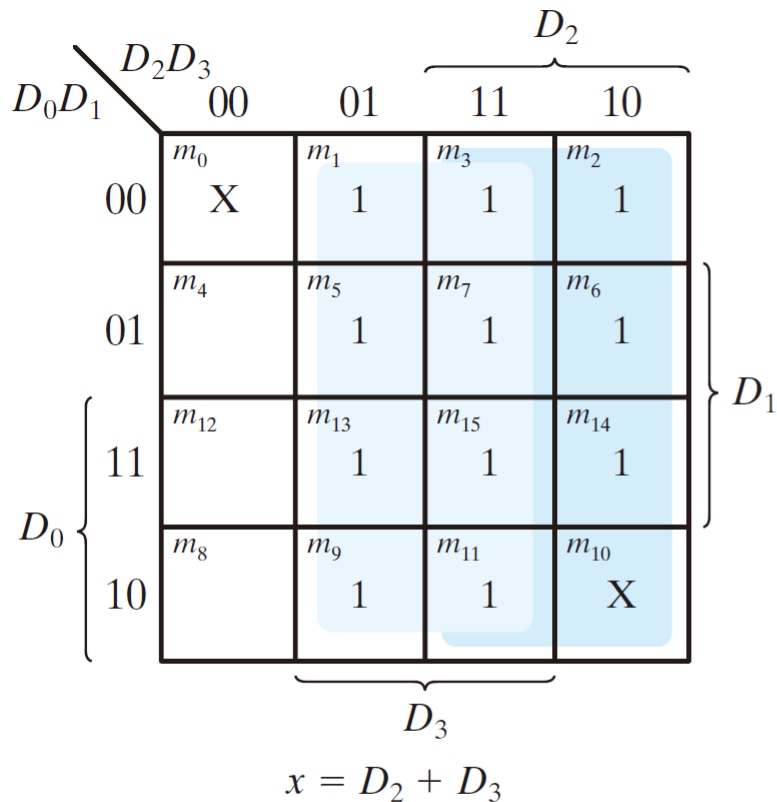
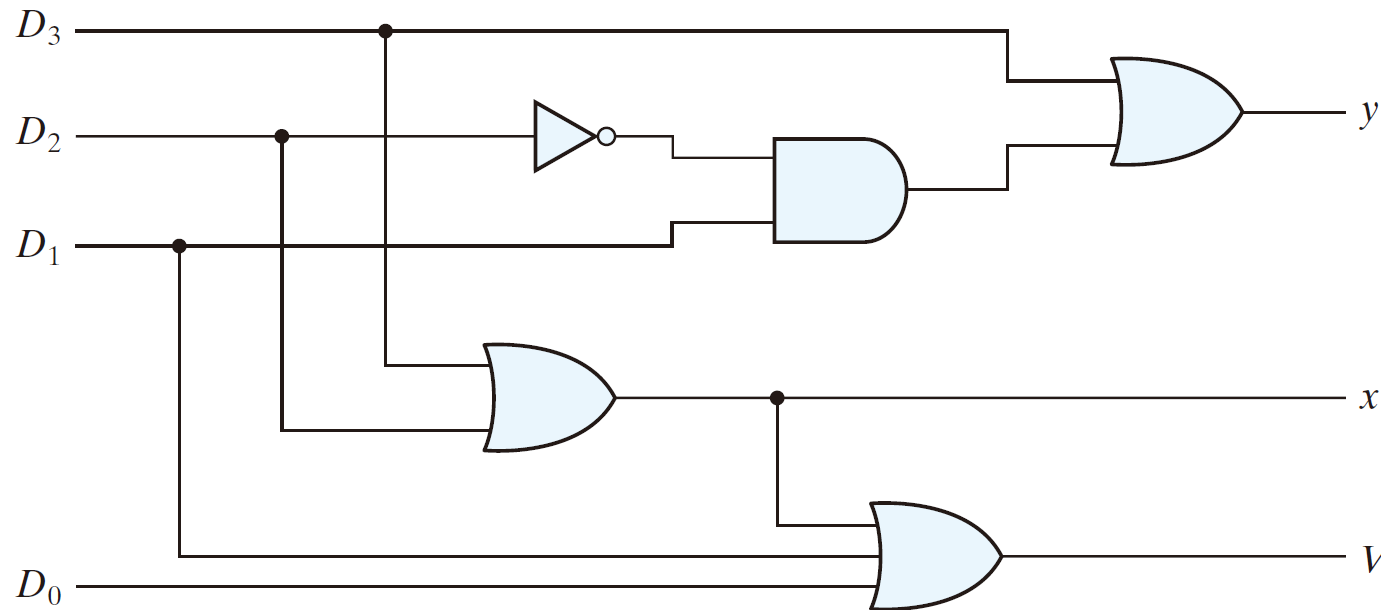


FIGURE 4.22

Maps for a priority encoder

4.10 Encoders (4/4)

- Implementation of priority



$$x = D_2 + D_3$$

$$y = D_3 + D_1 D_2'$$

$$V = D_0 + D_1 + D_2 + D_3$$

FIGURE 4.23
Four-input priority encoder

4.11 Multiplexers (1/10)

- Also known as MUX
- Select binary information from one of many input lines and direct it to a single output line
- 2^n input lines, n selection lines, and one output line
- Example: 2-to-1-line multiplexer

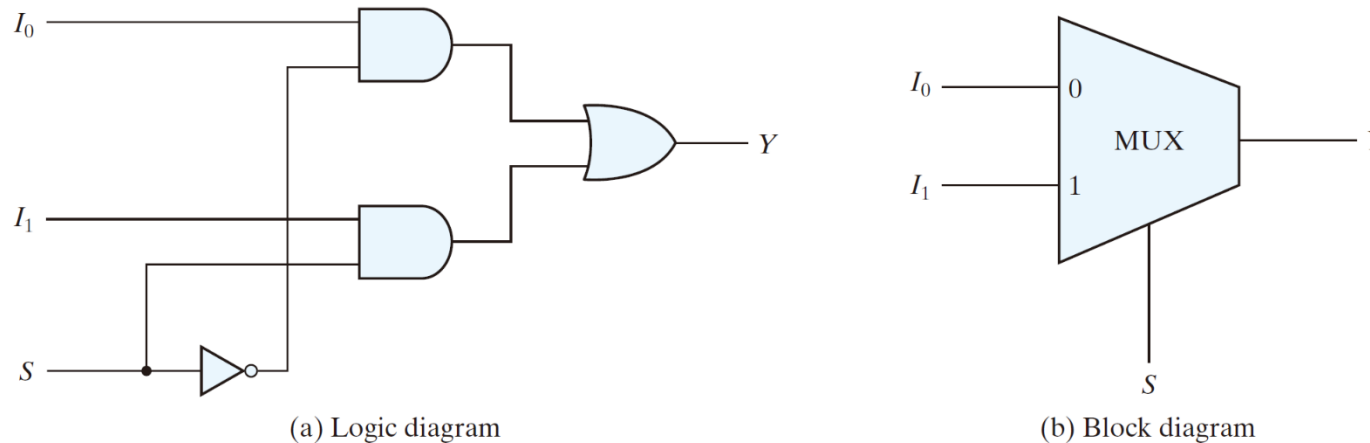


FIGURE 4.24
Two-to-one-line multiplexer

4.11 Multiplexers (2/10)

- 2^n -to-1-line MUX is constructed from
 - n -to- 2^n decoder
 - add each input line to each AND gate
 - OR all AND gates
 - an enable input (an option)

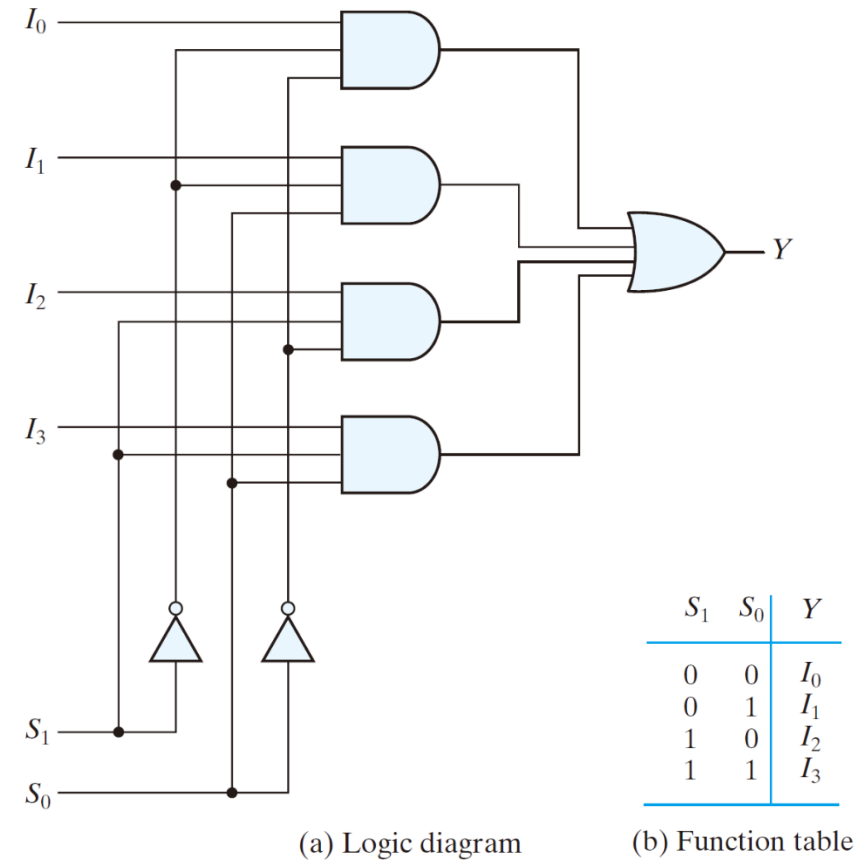
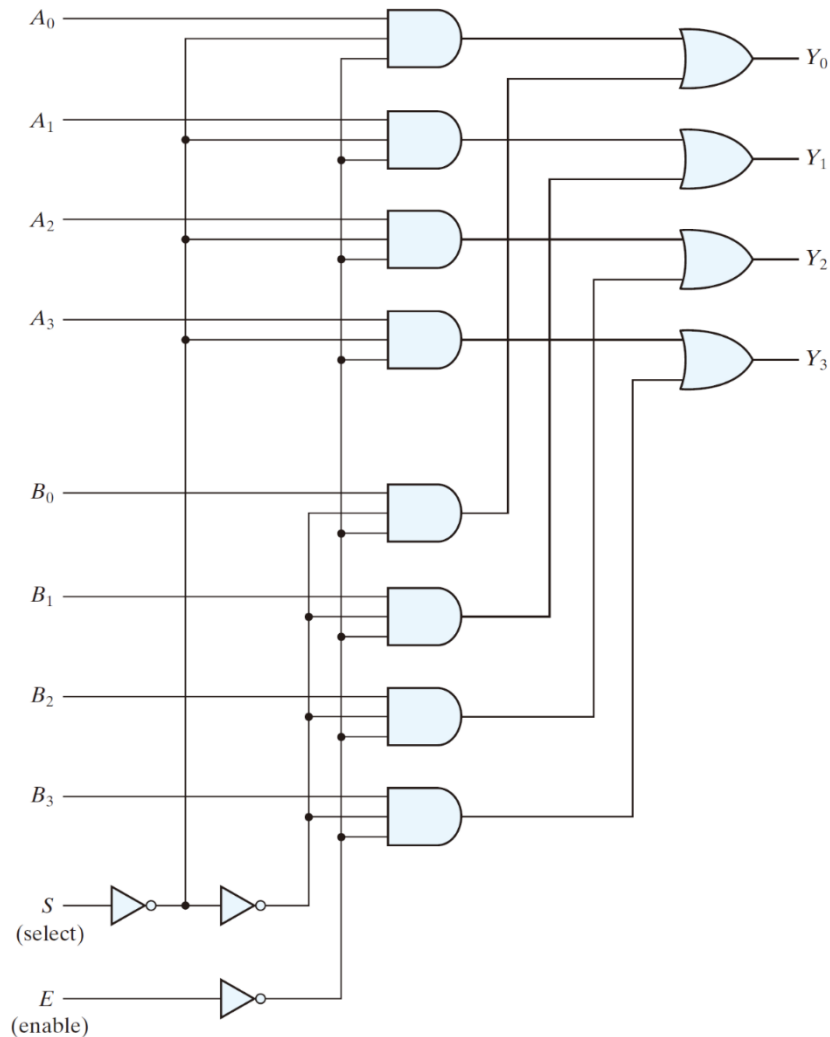


FIGURE 4.25
Four-to-one-line multiplexer

4.11 Multiplexers (3/10)

- Boolean function implementation
 - MUX: a decoder + an OR gate
 - 2^n -to-1 MUX can implement any Boolean function of n input variables
 - A better solution: implement any Boolean function of $n + 1$ input variables
 - n of these variables: the selection lines
 - the remaining variable: the input

4.11 Multiplexers (4/10)



E	S	Output Y
1	X	all 0's
0	0	select A
0	1	select B

Function table

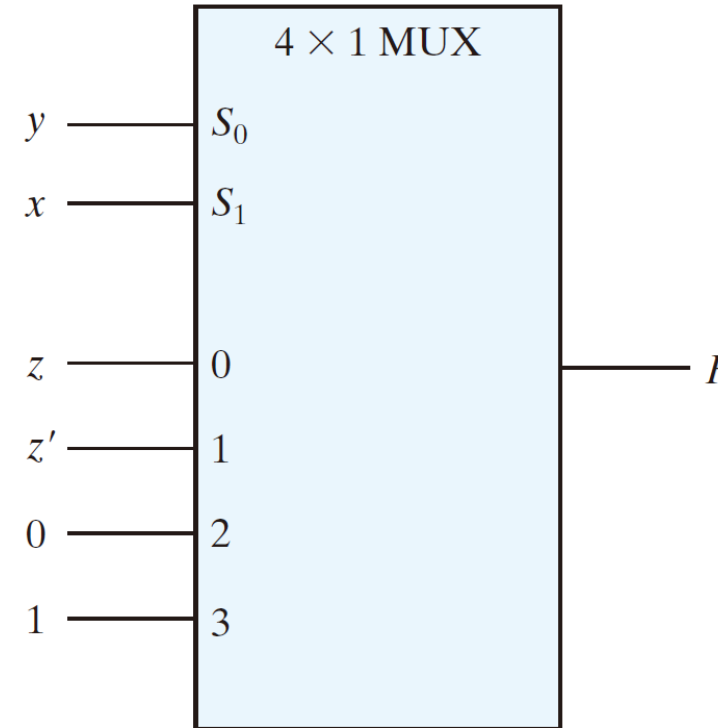
FIGURE 4.26
Quadruple two-to-one-line multiplexer

4.11 Multiplexers (5/10)

- Example: $F(x, y, z) = \Sigma(1, 2, 6, 7)$

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



(b) Multiplexer implementation

FIGURE 4.27

Implementing a Boolean function with a multiplexer

4.11 Multiplexers (6/10)

- Procedure
 - Assign an ordering sequence of the input variables
 - The rightmost variable will be used for the input lines
 - Assign the remaining $n-1$ variables to the selection lines w.r.t. (with respect to) their corresponding sequence
 - Construct the truth table
 - Consider a pair of consecutive minterms starting from m_0
 - Determine the input lines

4.11 Multiplexers (7/10)

- Example: $F(A, B, C, D) = \Sigma(1, 3, 4, 11, 12, 13, 14, 15)$

<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>F</i>	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	

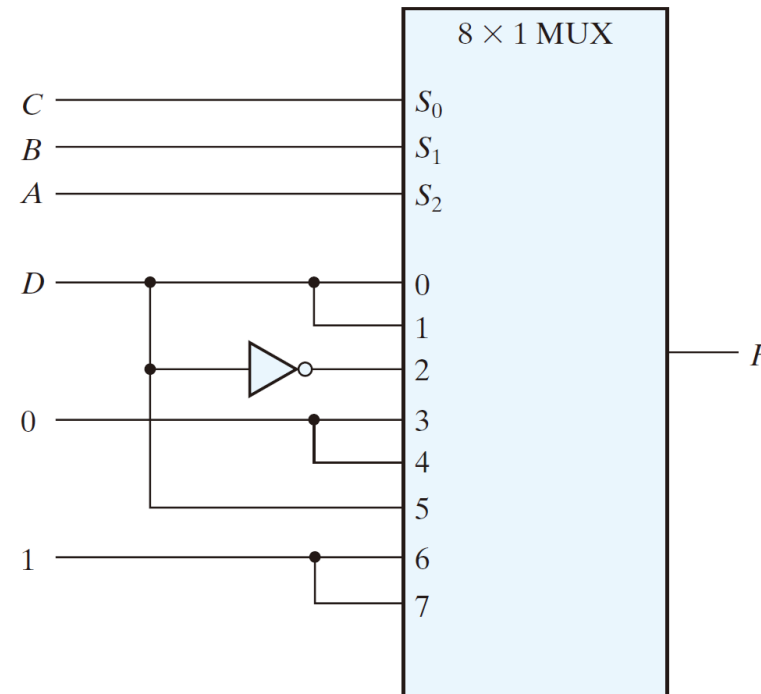


FIGURE 4.28
Implementing a four-input function with a multiplexer

4.11 Multiplexers (8/10)

- **Practice Exercise 4.9** Implement the Boolean function $F(A, B, C) = \Sigma(3, 5, 6, 7)$ with a MUX.

Answer:

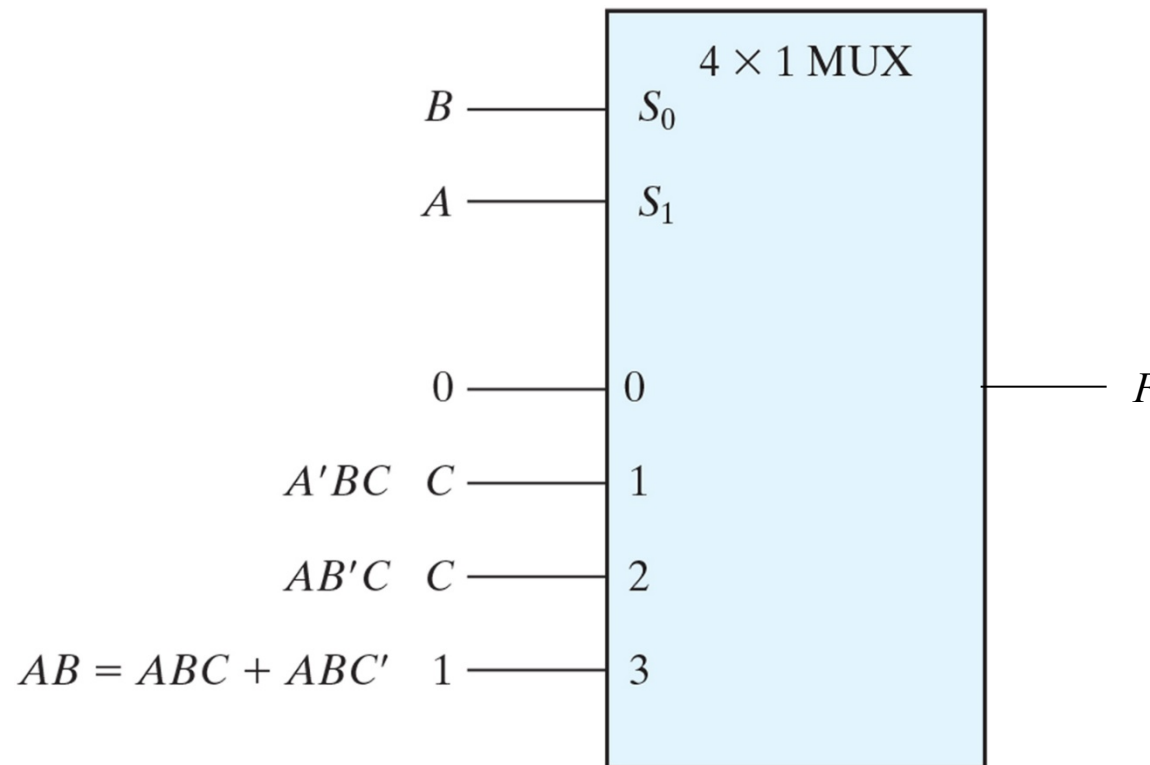


FIGURE PE4.9

4.11 Multiplexers (9/10)

- Three-state gates
 - A multiplexer can be constructed with three-state gates
 - Output state: 0, 1, and high-impedance or open circuits (ckts)

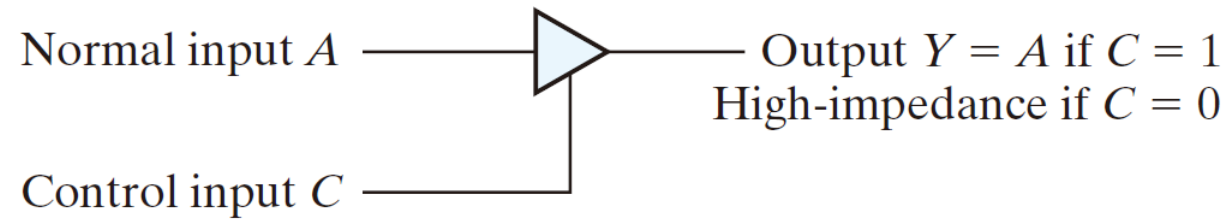


FIGURE 4.29
Graphic symbol for a three-state buffer

4.11 Multiplexers (10/10)

- Example: 2-to-1-line and 4-to-1-line multiplexers

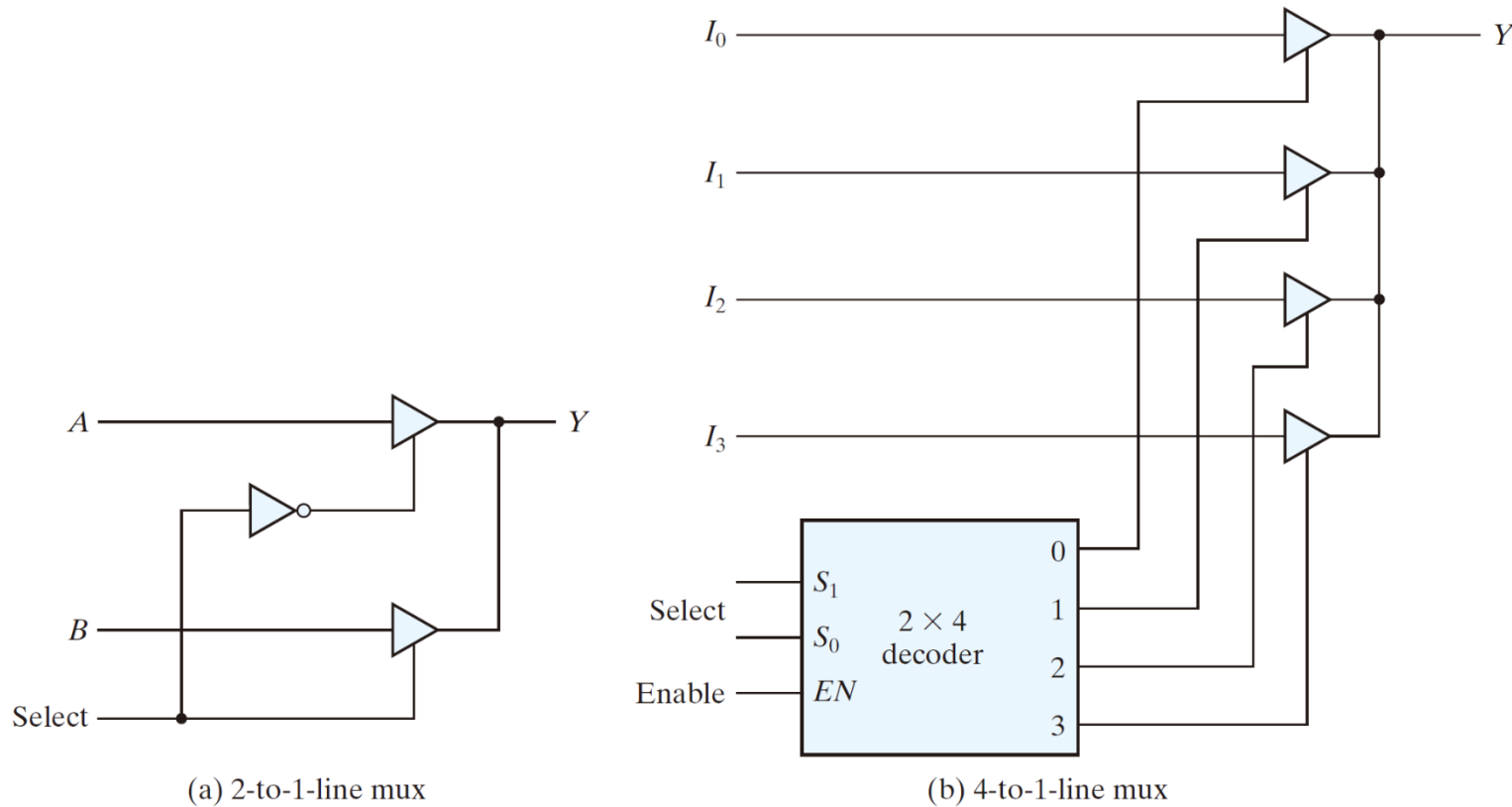


FIGURE 4.30
Multiplexers with three-state gates

4.12 HDL Models of Combinational Circuits (1/18)

- Three common styles to model combinational circuits
 - Gate-level modeling (or structural modeling) uses instantiations of predefined and user-defined primitive gates.
 - Dataflow modeling uses HDL operators and continuous assignment statements to describe the functionality represented by the Boolean function.
 - Behavioral modeling uses language-specific procedural statements to form an abstract model of a circuit.
- *Verilog*—Construct corresponding to each of three “classical” approaches to designing combinational logics
 - Continuous assignments (Boolean equations)
 - Built-in primitives (logic diagrams)
 - User-defined primitives (truth tables)

4.12 HDL Models of Combinational Circuits (2/18)

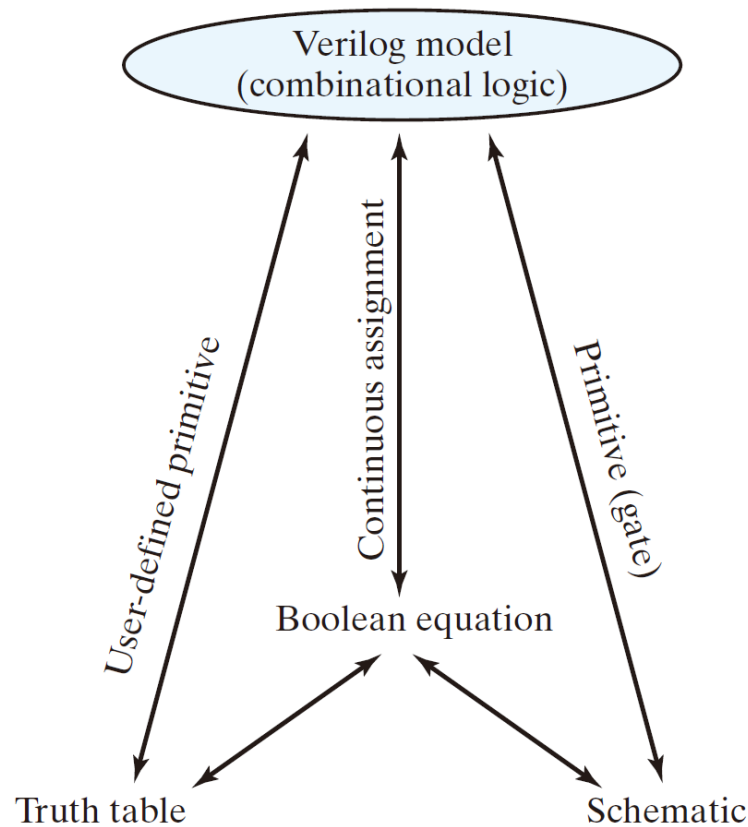


FIGURE 4.31

Relationship of Verilog constructs to truth tables, Boolean equations, and schematics

4.12 HDL Models of Combinational Circuits (3/18)

Gate-Level Modelling

- The four-valued logic truth tables for the **and**, **or**, **xor**, and **not** primitives

Table 4.9
Truth Table for Predefined Primitive Gates

and	0	1	x	z	or	0	1	x	z
0	0	0	0	0	0	0	1	x	x
1	0	1	x	x	1	1	1	1	1
x	0	x	x	x	x	x	1	x	x
z	0	x	x	x	z	x	1	x	x

xor	0	1	x	z	not	input	output
0	0	1	x	x		0	1
1	1	0	x	x		1	0
x	x	x	x	x		x	x
z	x	x	x	x		z	x

4.12 HDL Models of Combinational Circuits (4/18)

Gate-Level Modelling



- Example:

output [0: 3] *D*;

wire [7: 0] *SUM*;

- The first statement declares an output vector *D* with four bits, 0 through 3.
- The second declares a wire vector *SUM* with eight bits numbered 7 through 0.

4.12 HDL Models of Combinational Circuits (5/18)

Gate-Level Modelling

- **HDL Example 4.1 Two-to-Four-line decoder**

HDL Example 4.1 (Two-to-Four-Line Decoder)

```
// Gate-level description of two-to-four-line decoder
// Refer to Fig. 4.19 with symbol E replaced by enable, for clarity.

module decoder_2x4_gates (D, A, B, enable);
  output      [0: 3]      D;
  input       A, B;
  input       enable;
  wire        A_not,B_not, enable_not;

  not
    G1 (A_not, A),
    G2 (B_not, B),
    G3 (enable_not, enable);
  nand
    G4 (D[0], A_not, B_not, enable_not),
    G5 (D[1], A_not, B, enable_not),
    G6 (D[2], A, B_not, enable_not),
    G7 (D[3], A, B, enable_not);

endmodule
```

4.12 HDL Models of Combinational Circuits (6/18)

Gate-Level Modelling

- **Practice Exercise 4.10 (Verilog)** Write a continuous assignment statement that is equivalent to the logic of *G4* in *decoder_2x4_gates*.
Answer: `assign D[0] = !(A) && (!B) && (!enable);`

4.12 HDL Models of Combinational Circuits (7/18)

HDL Models of Three-State Gates

- A three-state gate has a data signal input, a data signal output, and a control input.
- The control input determines if the gate is in its normal operating state or in its high-impedance state.
- Statement: *gate name (output, input, control)*
 - gate name can be **bufif1**, **bufif0**, **notif1**, **notif0**
 - output can be 0, 1, x, or z

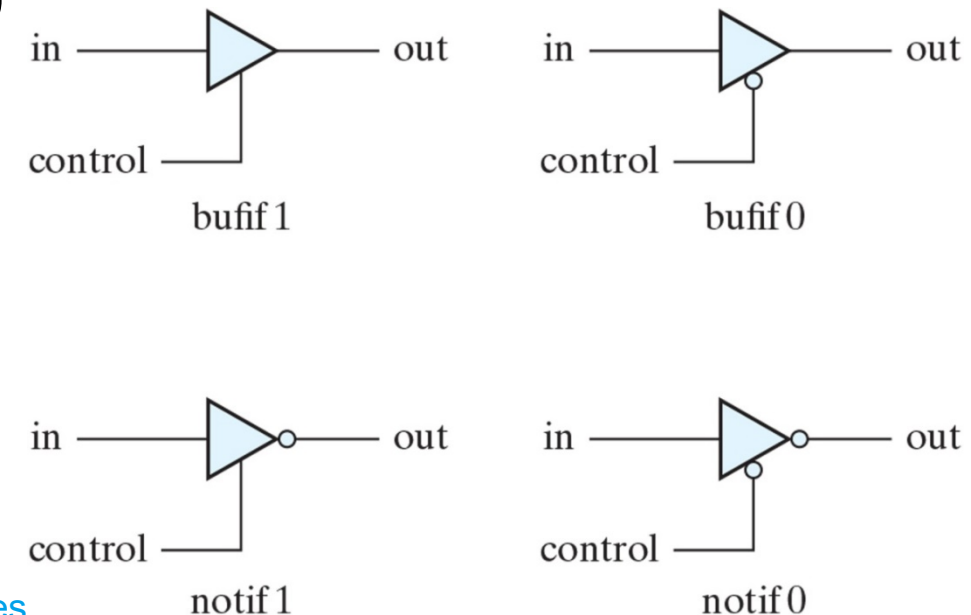


FIGURE 4.35
Three-state gates.

4.12 HDL Models of Combinational Circuits (8/18)

HDL Models of Three-State Gates

- Examples of gate instantiation
 - **bufif1** (OUT, A, control); //OUT = A when control = 1. OUT = **z** if control = 0.
 - **notif0** (Y, B, enable);

The HDL description must use a **tri** data type for the output:

// Mux with three-state output

```
module mux_tri (m_out, A, B, select);
```

```
  output m_out;
```

```
  input  A, B, select;
```

```
  tri    m_out;
```

```
  bufif1 (m_out, A, select);
```

```
  bufif0 (m_out, B, select);
```

```
endmodule
```

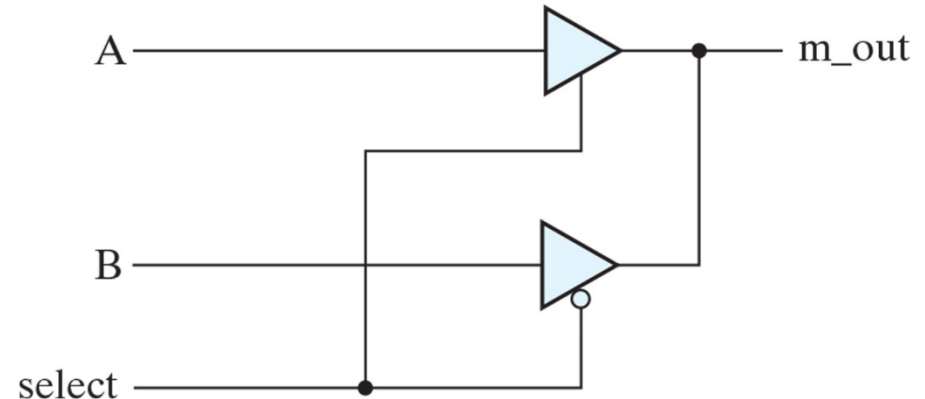


FIGURE 4.36

Two-to-one-line multiplexer with three-state buffers.

4.12 HDL Models of Combinational Circuits (9/18)

HDL Models of Three-State Gates

- The data type *net* contains
 - **wire**, **wor**, **wand**, **tri**, **supply1**, **supply0**
 - An identifier is used without being declared, it is default regarded as a **wire**.
 - **wor** models the hardware implementation of the wired-OR configuration
 - **wand** models the wired-AND configuration (open-collector, see Fig. 3.26)
 - **supply1** and **supply0** represent power supply and ground, respectively and are used to hardwire an input of a device to either 1 or 0.

4.12 HDL Models of Combinational Circuits (10/18)



HDL Models of Three-State Gates

- **Practice Exercise 4.11** Describe the functionality of a three-state inverter.
Answer: The output signal of a three-state inverter is the complement of the input signal if the inverter is enabled. If a three-state inverter is not enabled, the output is the high impedance state.

4.12 HDL Models of Combinational Circuits (11/18)

Number Representation

- Numbers in HDLs are represented in the format $n'Bv$
 - n is the number of bits (or size) used to store the value
 - If the size of the number is not specified, the size of the number is assumed at least 32 bits.
 - The integer data type (keyword **integer**) is stored in a 32-bit representation.
 - B is the base for interpreting the value.
 - If the base is not specified, the number is, by default, to be interpreted as a decimal value.
 - Numbers are stored in a bit pattern in memory.
 - $'b$, $'d$, $'o$, $'h$ are used to represent binary, decimal, octal, and hexadecimal, e.g., $4'hA = 4'd10 = 4'b1010$
 - v is the value to be interpreted and stored.
 - If the specified size exceeds the number of bits needed to represent the interpreted value, 0s are used to pad the number to full size.
 - A variable that is assigned $'0$ gets all 0s.
 - The underscore ($_$) may be inserted in a number to improve the readability, e.g., $8'b0101_1110$.

4.12 HDL Models of Combinational Circuits (12/18)

Number Representation



- **HDL Example 4.3 (Number Representation) Verilog**
 - 3'b110 stores in 3 bits the binary equivalent of decimal 6.
 - 8'hA5 stores in 8 bits the binary equivalent of hexadecimal A5H = 1010_01012 = 16510.
 - 8'b101 stores in 8 bits the binary value 0000_0101. Note the padding with 0s.
- **Practice Exercise 4.13** What is the binary word that will be stored for A = B5_H?
Answer: 10110101

4.12 HDL Models of Combinational Circuits (13/18)

Dataflow Modeling

- Dataflow models describe combinational circuits by their *function* rather than by their gate structure.
- A common form of dataflow modeling of combinational logic uses concurrent signal assignment statements and built-in language operators to express how signals are assigned values.
- Verilog has two families of predefined data types: **newts** and **variables** (also referred to as registers)
 - The net family includes **wire**, **wor**, **wand**, **tri**, **supply1**, **supply0**.
 - The variable family includes **reg**, **integer**, and **time**.
 - A **reg** may be a scalar or a vector quantity.
 - An **integer** is sized to the word length of the host machine and is at least 32 bits wide.
 - A **time** is represented by an unsigned 64-bit quantity.

4.12 HDL Models of Combinational Circuits (14/18)

Dataflow Modeling

- Verilog provides about 30 different operators.
- Example: **assign** Y = (A && S) || (B && (!S))

Table 4.10
Some Verilog HDL Operators

Symbol	Operation	Symbol	Operation
+	binary addition		
−	binary subtraction		
&	bitwise AND	&&	logical AND
	bitwise OR		logical OR
^	bitwise XOR		
~	bitwise NOT	!	logical NOT
= =	equality		
>	greater than		
<	less than		
{ }	concatenation		
?:	conditional		

4.12 HDL Models of Combinational Circuits (15/18)

Dataflow Modeling

HDL Example 4.4 (Dataflow: Two-to-Four Line Decoder)

Verilog

```
// Dataflow description of two-to-four-line decoder
// See Fig. 4.19. Note: The figure uses symbol E, but the
// Verilog model uses enable to clearly indicate functionality.
```

```
module decoder_2x4_df ( // Verilog 2001, 2005 syntax
    output [0: 3]      D,
    input              A, B,
                      enable
);
    assign D[0] = !((!A) && (!B) && (!enable)),
           D[1] = !((!A) && B && (!enable)),
           D[2] = ((A) && (! B) && (!enable)),
           D[3] = !(A && B && (!enable));
endmodule
```

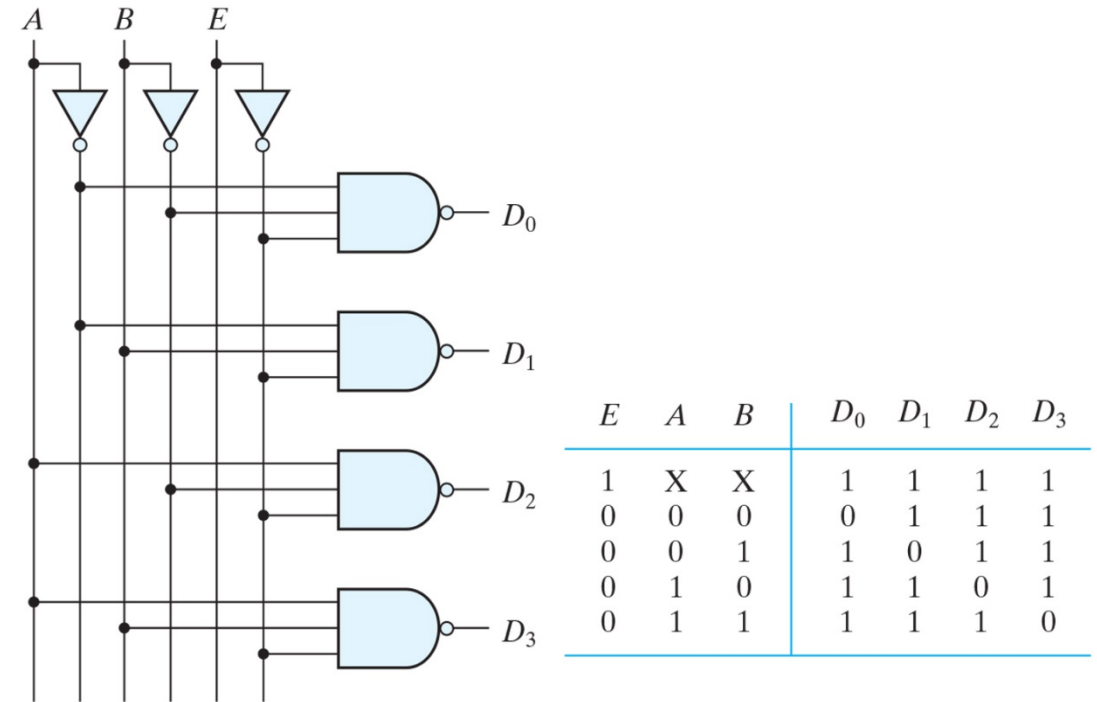


FIGURE 4.19

Two-to-four-line decoder with enable input.

4.12 HDL Models of Combinational Circuits (16/18)

Dataflow Modeling



HDL Example 4.5 (Dataflow: Four-Bit Adder)

Verilog

```
// Dataflow description of four-bit adder
// Verilog 2001, 2005 module port syntax
module binary_adder (
    output      C_out,
    output [3: 0] Sum,
    input [3: 0] A, B,
    input      C_in
);
    assign {C_out, Sum} = A + B + C_in    // Continuous assignment statement
endmodule
```

← {C_out, Sum} represents the five-bit result of the addition operation.

4.12 HDL Models of Combinational Circuits (17/18)

Dataflow Modeling

HDL Example 4.6 (Dataflow: Four-Bit Comparator)

A 4-bit magnitude comparator has two 4-bit inputs A and B and three outputs. One output (A_lt_B) is logic 1 if A is less than B , a second output (A_gt_B) is logic 1 if A is greater than B , and a third output (A_eq_B) is logic 1 if A is equal to B .

Verilog

```
// Dataflow description of a four-bit comparator // V2001, 2005, SV syntax

module mag_compare
  (output A_lt_B, A_eq_B, A_gt_B,
   input [3:0] A, B
  );
  assign A_lt_B = (A < B);           // Continuous assignment statements
  assign A_gt_B = (A > B);
  assign A_eq_B = (A == B);
endmodule
```

4.12 HDL Models of Combinational Circuits (18/18)

Dataflow Modeling

- A Verilog conditional operator takes three operands

$$condition ? true_expression : false_expression;$$
- Example: **assign** OUT = select ? A : B

HDL Example 4.7 (Dataflow: Two-to-One Multiplexer)

Verilog

```
// Dataflow description of two-to-one-line multiplexer
module mux_2x1_df (m_out, A, B, select);
output      m_out;
input       A, B;
input       select;
    assign m_out = (select)? A : B;    // Conditional operator
endmodule
```

4.13 Behavioral Modeling (1/5)

- Behavioral modeling represents digital circuits at a functional and algorithmic level.
- Behavioral models execute one or more procedural statements when launched by a sensitivity mechanism, commonly called *a sensitivity list*, which monitors signals and launches the execution of the behavioral description.
- Procedural statements are assignments and statements that control the sequence of execution, for example, **do**, **loop**, **case**, and **if** statements.

4.13 Behavioral Modeling (2/5)

Verilog (Procedural Assignment Statements)

- Verilog behavioral descriptions of hardware are declared with the keyword **always**, followed by an optional event control expression (sensitivity list) and a **begin ... end** block of procedural assignment statements.
- Verilog has two types of assignment statements: *continuous* and *procedural*.
- Procedural assignment statements
 - are those made within the scope of an **always** or **initial** procedural statement.
 - **always** statement executes repeatedly, as a loop.
 - **initial** statement executes only once.
 - use the blocking assignment operator “=” if the assignment represents sequential behavior.
 - use the nonblocking assignment operator “<=” if the assignment represents concurrent behavior.
- The event control expression in a procedural statement
 - effectively specifies *when* the associated statements will begin to execute because it suspends execution of the procedural statement until one or more of the signals in the expression has an event (qualifies or otherwise).
 - In its absence, the associated statements begin execution immediately at the beginning of simulation.

4.13 Behavioral Modeling (3/5)

Verilog (Procedural Assignment Statements)

HDL Example 4.8 (Behavioral: Alternative Two-to-Four Line Decoder)

Verilog

```

module decoder_2x4_df_beh ( // Verilog 2001, 2005 syntax
    output [0: 3] D,
    input      A, B,
                enable
);
always @ (A, B, enable) begin
    D[0] <= !((!A) && (!B) && (!enable)),
    D[1] <= !((!A) && B && (!enable)),
    D[2] <= !(A && (!B) && (!enable)),
    D[3] <= !(A && B && (!enable));
end;
endmodule

```

← event control expression or sensitivity list

1. Each line is a procedural assignment statement.
2. Procedural assignment statements are executed whenever there is a change in any of the variables listed in the sensitivity list.
3. There is no semicolon at the end of **always** statement.

4.13 Behavioral Modeling (4/5)

Verilog (Procedural Assignment Statements)

HDL Example 4.9 (Behavioral: Two-to-One Line Multiplexer)

Verilog (Procedural Statement)

// Behavioral description of two-to-one-line multiplexer

```
module mux_2x1_beh (m_out, A, B, select);
```

```
    output    m_out;
```

```
    input     A, B, select;
```

```
    reg       m_out; ←
```

```
    always @ (A or B or select)    // Alternative: always @ (A, B, select)
```

```
    if (select == 1) m_out = A;    // Alternative: if (select) m_out = A;
```

```
    else m_out = B;
```

```
endmodule
```

The signal m_out must be of the **reg** data type because it is assigned value by a Verilog procedural assignment statement. A **reg** data type is not necessarily monitored.

4.13 Behavioral Modeling (5/5)

Verilog (Procedural Assignment Statements)

HDL Example 4.10 (Behavioral: Four-to-One Line Multiplexer)

This example provides behavioral descriptions of a four-to-one-line multiplexer. A two-bit vector input, *select*, determines which of the four input channels provides value to the output.

Verilog

```
// Behavioral description of four-to-one line multiplexer
// Verilog 2001, 2005 port syntax
module mux_4x1_beh
(output reg m_out,
 input in_0, in_1, in_2, in_3,
 input [1: 0] select
);
always @ (in_0, in_1, in_2, in_3, select) // Verilog 2001, 2005, SV syntax
case (select)
2'b00: m_out <= in_0;
2'b01: m_out <= in_1;
2'b10: m_out <= in_2;
2'b11: m_out <= in_3;
endcase
endmodule
```

Two variants of **case** construct: **casex** and **casez**

The **casex** construct treats don't cares any bits of the **case** expression or the **case** item that have logic value **x** or **z**.

The **casez** construct treats as don't cares only the logic value **z** for the purpose of detecting a match between the **case** expression and a **case** item.

1. In the absence of a match, no statement is executed.
2. The list of **case** items need not be complete.
3. A **default** case item and an associated case expression can be included in the sensitivity list to ensure that a statement will always be executed.

4.14 Writing A Simple Testbench (1/9)

- A testbench is an HDL program that describes and applies a stimulus to an HDL model of a circuit to test it and observe its response during simulation.
- Testbench must be carefully designed to write stimuli that will test a circuit thoroughly, exercising all of the operating features that are specified.
- In Verilog, testbenches use the **initial** statement to provide a stimulus to the circuit being tested.
- The **initial** statement
 - executes starting from time 0, and may continue executing with any assignments that are delayed by a given number of time units, as specified by the symbol #.
 - expires when the last statement in its block executes, which may or may not coincide with the end of the simulation.

4.14 Writing A Simple Testbench (2/9)

- Example

initial

begin

A = 0; B = 0;

#10 A = 1;

#20 A = 0; B = 1;

end

- Example

initial

begin

D = 3'b000;

repeat (7)

#10 D = D + 3'b001;

end

4.14 Writing A Simple Testbench (3/9)

- A simple stimulus module has the following form:

```
module test_module_name;
    // Declare local reg and wire identifiers.
    // Instantiate the design module under test.
    // Specify a stopwatch, using $finish to terminate the simulation.
    // Generate stimulus, using initial and always statements.
    // Display the output response (text or graphics (or both)).
endmodule
```

4.14 Writing A Simple Testbench (4/9)

- A test module is, in fact, a module that has no inputs or outputs.
- The signals that are applied as inputs to the unit under test (UUT) for simulation are declared in the stimulus module as local **reg** data type.
- Each output of the design module that is displayed for testing is declared in the stimulus module as a local **wire** data type.
- The module under test is then instantiated, using the local identifiers in its port list.
- The stimulus module generates inputs for the design module by declaring local identifiers t_A and t_B as **reg** data type and checks the output of the design unit with the **wire** identifier t_C.
- The local identifiers are then used to stimulate the design module being tested.
- The simulator associated the (actual) local identifiers of the inputs within the testbench, t_A, t_B, and t_C, with the formal identifiers of the module (A, B, and C).

4.14 Writing A Simple Testbench (5/9)

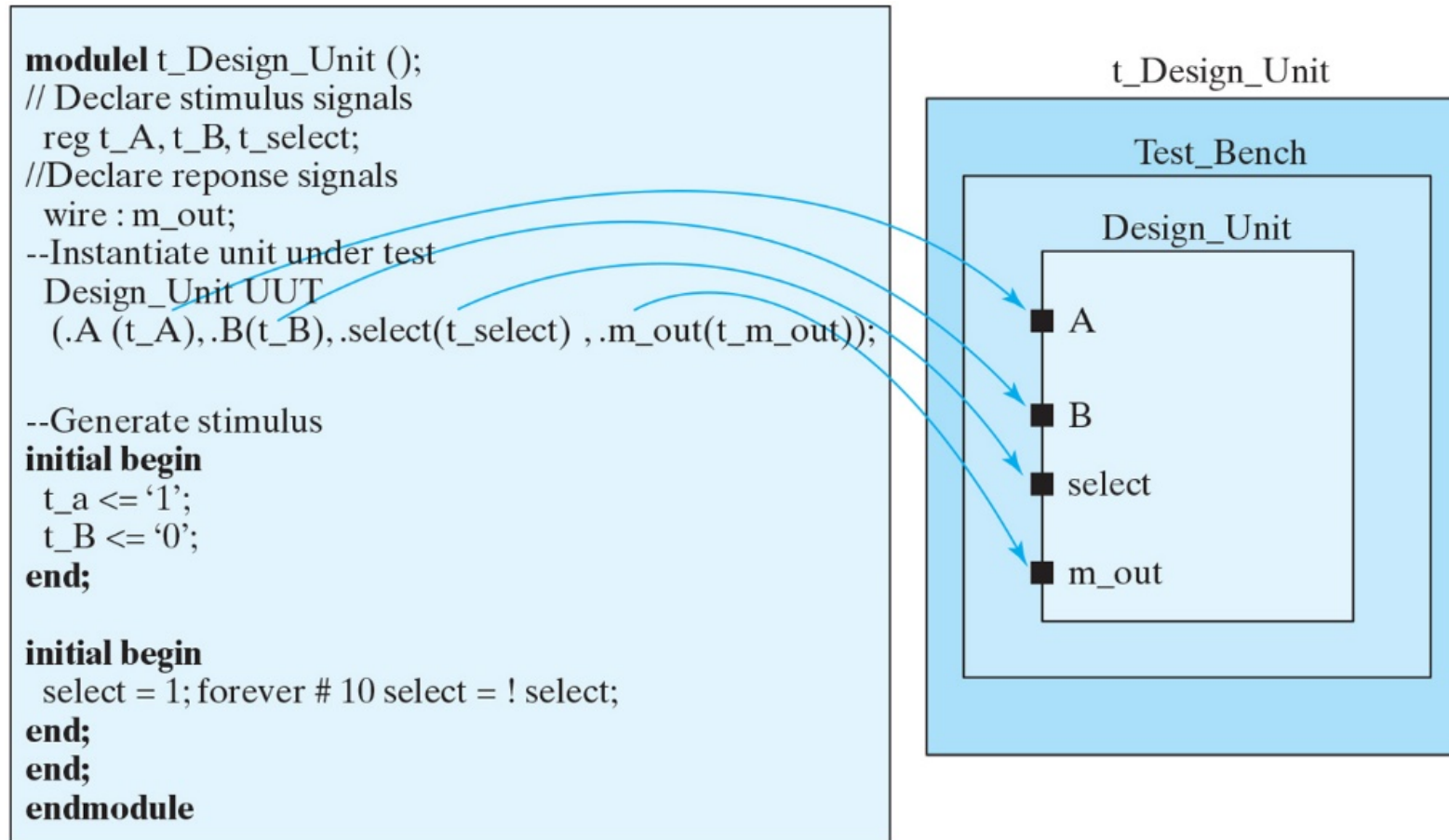


FIGURE 4.37

Interaction between testbench and Verilog design unit.

4.14 Writing A Simple Testbench (6/9)

- The response to the stimulus generated by the **initial** and **always** blocks will appear in text format as standard output and as waveforms (timing diagrams) in simulators having graphical output capability.
- Numerical outputs are displayed by using Verilog system tasks that are recognized by keywords that begin with the symbol **\$**.
 - **\$display** --- display a one-time value of variables or strings with an end-of-line return
 - **\$write** --- same as **\$display**, but without going to the next line,
 - **\$monitor** --- display variables whenever a value changes during the simulation run,
 - **\$time** --- display the simulation time, and
 - **\$finish** --- terminate the simulation.
- The syntax for **\$display**, **\$write**, and **\$monitor** is of the form

Task-name (format specification, argumentlist);

4.14 Writing A Simple Testbench (7/9)

- The format specification uses the symbol % to specify the radix of the numbers that are displayed and may have a string enclosed in quoted (").
- The base may be binary, decimal, hexadecimal, or octal, identified with the symbols %b(B), %d(D), %h(H), and %o(O), respectively.
 - Example: **\$display** ("%d %b %b", C, A, B);
 - Example: **\$display** ("time = %0d A = %b B = %b", **\$time**, A, B);
 - Produce the display time = 3 A = 10 B = 1
 - %0d provides a display of the significant digits without the leading spaces

4.14 Writing A Simple Testbench (8/9)

HDL Example 4.11 (Testbench)

Verilog

```
// Testbench with stimulus for mux_2x1_df
module t_mux_2x1_df;
  wire    t_mux_out;
  reg     t_A, t_B;
  reg     t_sel;
  parameter stop_time = 50;
  mux_2x1_df M1 (t_mux_out, t_A, t_B, t_sel);    // Instantiation of circuit to be tested
  // Alternative association of ports by name:
  // mux_2x1_df M1 (.mux_out (t_mux_out), .A(t_A), .B(t_B), .select(t_sel));

  initial # stop_time $finish;
  initial begin                                // Stimulus generator
    t_sel = 1; t_A = 0; t_B = 1;
```


4.14 Writing A Simple Testbench (9/9)

```
#10 t_A = 1; t_B = 0;
#10 t_sel = 0;
#10 t_A = 0; t_B = 1;
end
initial begin                                // Response monitor
    // $display (" time Sel A B m_out ");
    // $monitor ($time,, " %b %b %b %b ", t_sel, t_A, t_B, t_mux_out);
    $monitor (" time = ", $time,, " t_sel = %b t_A = %b t_B = %b t_mux_out = %b",
        t_sel, t_A, t_B, t_mux_out);
end
endmodule

// Dataflow description of two-to-one-line multiplexer
// from Example 4.6
module mux_2x1_df (m_out, A, B, sel);
    output m_out;
    input A, B;
    input sel;
    assign m_out = (sel) ? A : B;
endmodule
```

Simulation log:

```
time = 0 | sel = 1 A = 0 B = 1 OUT = 0
time = 10 | sel = 1 A = 1 B = 0 OUT = 1
time = 20 | sel = 0 A = 1 B = 0 OUT = 0
time = 30 | sel = 0 A = 0 B = 1 OUT = 1
```

4.15 Logic Simulation (1/6)

- Logic simulation provides a fast and accurate method of verifying that a model of a combinational circuit is correct.
- There are two types of verification: functional and timing.
 - In functional verification, the logical operation of the circuit is verified by assuming zero-delay model, i.e., ignoring the propagation delay of physical gates.
 - In timing verification, operations of the circuit is verified by considering the effect of delays through gates.
- Logic simulation is usually accomplished with event-driven simulators.
- An event is said to occur in a sequential circuit when a signal undergoes a change in value.
- A simulation of a digital circuit is said to be “event-driven” when the activity of the simulator is initiated only at those times when the signals in the model experience a change.

4.15 Logic Simulation (2/6) With Zero Propagation Delay

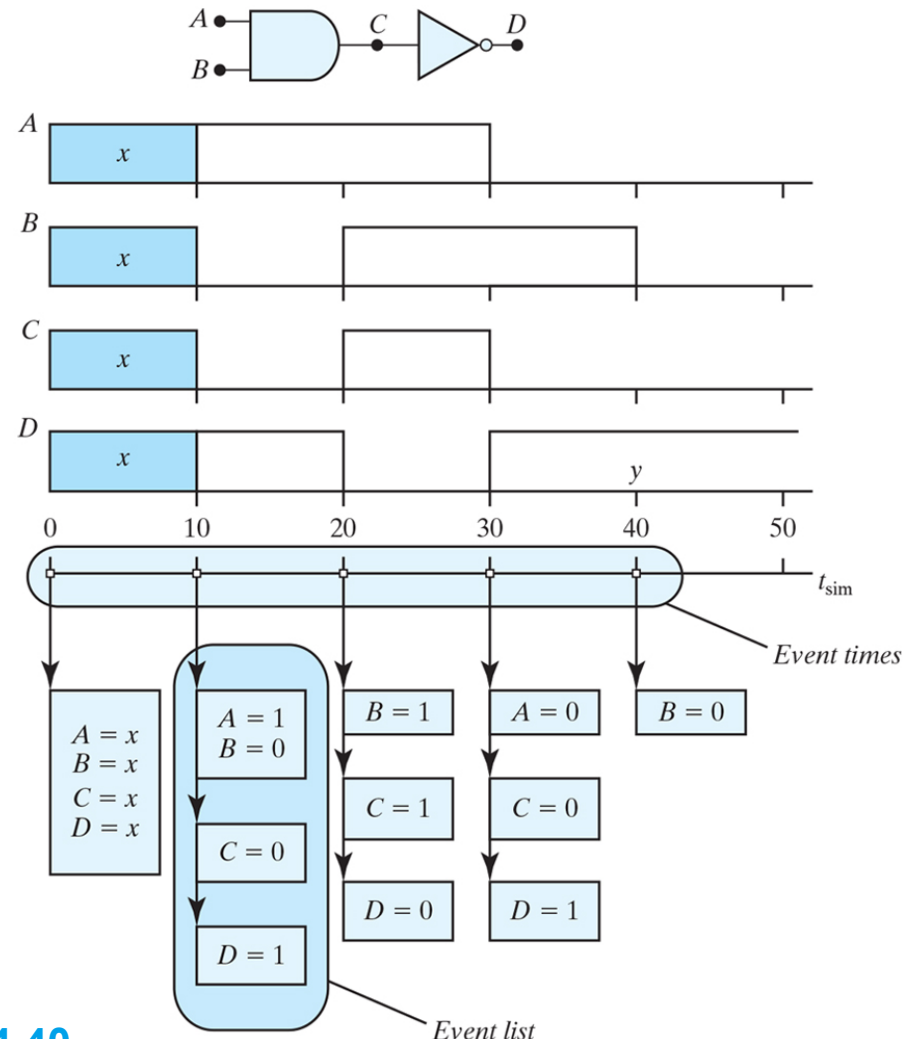


FIGURE 4.40

Representation of event-driven simulation (with zero delay).

4.15 Logic Simulation (3/6) With Propagation Delay

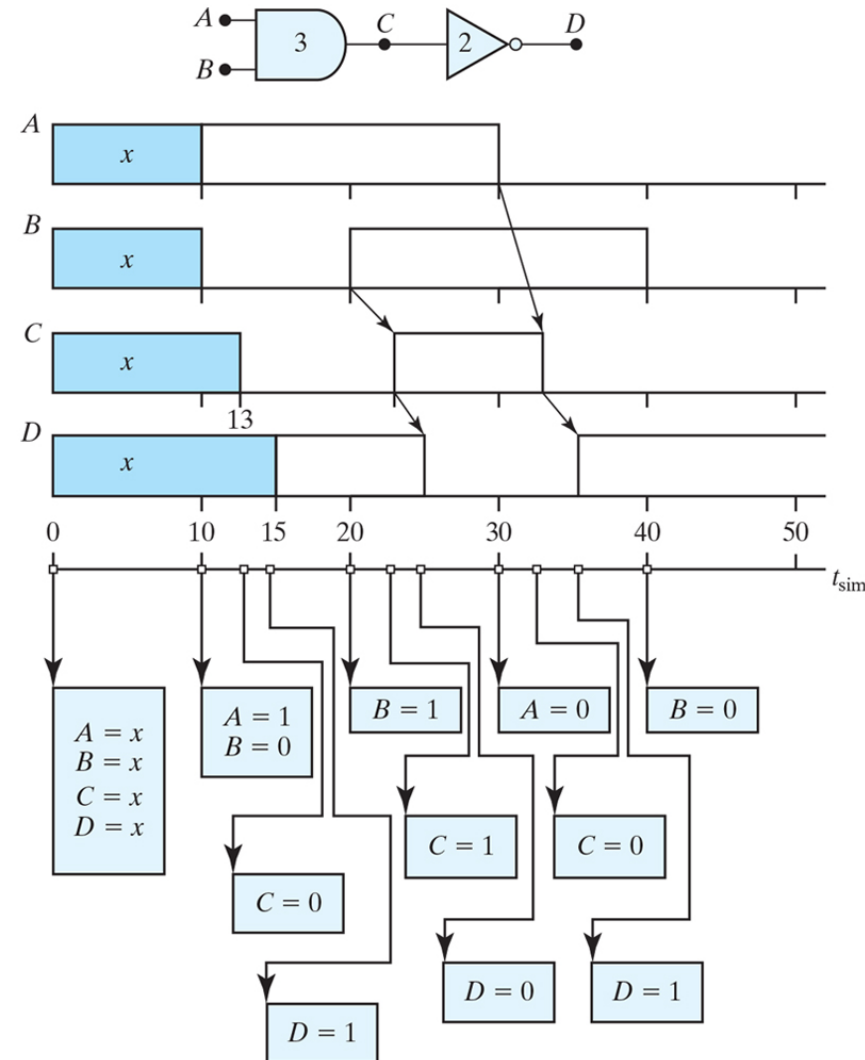


FIGURE 4.41

Representation of event-driven simulation (with propagation delay).

4.15 Logic Simulation (4/6)

HDL Example 4.13 (Logic Simulation)

Verilog

The stimulus for the circuit is listed in the second module. The inputs are specified with a three-bit **reg** vector D . $D[2]$ is equivalent to input A , $D[1]$ to input B , and $D[0]$ to input C . The outputs of the circuit F_1 and F_2 are declared as type **wire**. The complement of F_2 is named $F2_b$ to illustrate a common practice for designating the complement of a signal (instead of appending *_not*). The procedure follows the steps represented by Fig. 4.37. The **repeat** loop provides the seven binary numbers after 000 for the truth table. The result of the simulation generates the output truth table displayed with the example. The truth table listed shows that the circuit is a full adder.

4.15 Logic Simulation (5/6)

```
// Gate-level description of circuit of Fig. 4.2
module Circuit_of_Fig_4_2 (A, B, C, F1, F2);
    input A, B, C;
    output F1, F2;
    wire T1, T2, T3, F2_b, E1, E2, E3;
    or G1 (T1, A, B, C);
    and G2 (T2, A, B, C);
    and G3 (E1, A, B);
    and G4 (E2, A, C);
    and G5 (E3, B, C);
    or G6 (F2, E1, E2, E3);
    not G7 (F2_b, F2);
    and G8 (T3, T1, F2_b);
    or G9 (F1, T2, T3);
endmodule

// Stimulus to analyze the circuit
module test_circuit;
    reg [2: 0] D;
    wire F1, F2;
    Circuit_of_Fig_4_2 UUT (D[2], D[1], D[0], F1, F2); // Instantiate UUT
```

4.15 Logic Simulation (6/6)

```

initial
  begin // Apply stimulus
    D = 3'b000;
    repeat (7) #10 D = D + 1'b1;
  end
  initial $monitor (" ABC = %b F1 = %b F2 = %b", D, F1, F2); // Observe response
endmodule

```

Simulation log:

```

ABC = 000, F1 = 0 F2 = 0
ABC = 001 F1 = 1 F2 = 0 ABC = 010 F1 = 1 F2 = 0
ABC = 011 F1 = 0 F2 = 1 ABC = 100 F1 = 1 F2 = 0
ABC = 101 F1 = 0 F2 = 1 ABC = 110 F1 = 0 F2 = 1
ABC = 111 F1 = 1 F2 = 1

```
